TABLE OF CONTENTS

Customer.java - User Account Management

File Location: `src/main/java/com/smarthome/model/Customer.java`

Overview

The Customer class is like a blueprint for creating customer accounts in a smart home system. Each customer can own devices, be part of family groups, and have security features to protect their account.

Package Declaration and Imports

```
package com.smarthome.model;

import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbBean;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbPartitionKey;
import software.amazon.awssdk.enhanced.dynamodb.mapper.annotations.DynamoDbAttribute;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
```

Explanation:

- Package declaration: Organizes code into folders (like file directories)
- DynamoDB imports: Help save customer data to a NoSQL database
- LocalDateTime: Handles dates and times
- ArrayList and List: Create and manage lists of items

Class Declaration

java

```
@DynamoDbBean
public class Customer {
```

Explanation:

- `@DynamoDbBean` tells the database this class can be saved as data
- `public class Customer` creates a class that other parts of the program can use
- This class serves as a template for creating customer objects

Instance Variables

java

```
private String email;
private String fullName;
private String password;
private List<Gadget> gadgets;
private List<String> groupMembers;
private String groupCreator;
private List<DeletedDeviceEnergyRecord> deletedDeviceEnergyRecords;
private List<DevicePermission> devicePermissions;
private int failedLoginAttempts;
private LocalDateTime accountLockedUntil;
private LocalDateTime lastFailedLoginTime;
```

Variable Purposes:

Variable	Type	Purpose
email	String	Customer's email address (unique identifier)
fullName	String	Customer's display name
password	String	Encrypted password for security
gadgets	List<Gadget>	Smart devices owned by customer
groupMembers	List<String>	Family members with device access
groupCreator	String	Who created the family group
deletedDeviceEnergyRecords	List	Energy data from deleted devices
devicePermissions	List	Controls who can use which devices

failedLoginAttempts	int	Security: counts failed login attempts
accountLockedUntil	LocalDateTime	Security: when account unlocks
lastFailedLoginTime	LocalDateTime	Security: timestamp of last failure

Constructors

Default Constructor

java

```
public Customer() {
    this.gadgets = new ArrayList<>();
    this.groupMembers = new ArrayList<>();
    this.groupCreator = null;
    this.deletedDeviceEnergyRecords = new ArrayList<>();
    this.devicePermissions = new ArrayList<>();
    this.failedLoginAttempts = 0;
    this.accountLockedUntil = null;
    this.lastFailedLoginTime = null;
}
```

Purpose: Creates an empty customer object with initialized collections to prevent errors.

Parameterized Constructor

java

```
public Customer(String email, String fullName, String password) {
    this.email = email;
    this.fullName = fullName;
    this.password = password;
    // Initialize all collections (same as default constructor)
    this.gadgets = new ArrayList<>();
    this.groupMembers = new ArrayList<>();
    this.groupCreator = null;
    this.deletedDeviceEnergyRecords = new ArrayList<>();
    this.devicePermissions = new ArrayList<>();
    this.failedLoginAttempts = 0;
    this.accountLockedUntil = null;
    this.lastFailedLoginTime = null;
}
```

Purpose: Creates a customer with basic information provided, used for new account registration.

Key Methods

Email Management (Database Primary Key)

java

```
@DynamoDbPartitionKey
public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}
```

Special Note: The `@DynamoDbPartitionKey` annotation marks email as the primary key for database operations.

Device Management

java

```
public void addGadget(Gadget gadget) {
    if (this.gadgets == null) {
        this.gadgets = new ArrayList<>();
    }

    boolean exists = this.gadgets.stream()
            .anyMatch(g -> g.getType().equalsIgnoreCase(gadget.getType()) &&
                           g.getRoomName().equalsIgnoreCase(gadget.getRoomName()));

    if (!exists) {
```

```
        this.gadgets.add(gadget);
    }
}
```

Key Features:
- Null safety: Creates list if it doesn't exist
- Duplicate prevention: Checks if device already exists
- Case-insensitive comparison: Ignores capitalization differences
- Stream operations: Modern Java way to process collections

Security Methods
java

```
public boolean isAccountLocked() {
    return accountLockedUntil != null && LocalDateTime.now().isBefore(accountLockedUntil);
}

public void incrementFailedAttempts() {
    this.failedLoginAttempts++;
    this.lastFailedLoginTime = LocalDateTime.now();
}

public void resetFailedAttempts() {
    this.failedLoginAttempts = 0;
    this.accountLockedUntil = null;
    this.lastFailedLoginTime = null;
}

public void lockAccount(int minutes) {
    this.accountLockedUntil = LocalDateTime.now().plusMinutes(minutes);
}
```

Security Features:
- Account locking: Prevents brute force password attacks
- Failed attempt tracking: Monitors suspicious activity
- Automatic unlocking: Accounts unlock after specified time
- Audit trail: Records when attempts occurred

Group Management
java

```
public void addGroupMember(String memberEmail) {
    if (this.groupMembers == null) {
        this.groupMembers = new ArrayList<>();
    }

    if (!this.groupMembers.contains(memberEmail.toLowerCase().trim())) {
        this.groupMembers.add(memberEmail.toLowerCase().trim());
    }
}


public boolean isGroupAdmin() {
   return this.groupCreator != null && this.groupCreator.equalsIgnoreCase(this.email);
}
```

Features:
- Family sharing: Multiple users can access shared devices
- Admin privileges: Group creator has special permissions
- Duplicate prevention: Same member can't be added twice
- Data normalization: Emails stored in consistent format

DeletedDeviceEnergyRecord.java - Energy Data Preservation

File Location: `src/main/java/com/smarthome/model/DeletedDeviceEnergyRecord.java`

Overview

When smart home devices are removed, their energy usage data shouldn't be lost. This class acts as a "digital receipt" preserving important energy consumption information for billing and historical analysis.

Class Structure

java

```
@DynamoDbBean
public class DeletedDeviceEnergyRecord {
    private String deviceType;
    private String roomName;
    private String deviceModel;
    private double totalEnergyConsumedKWh;
    private long totalUsageMinutes;
    private LocalDateTime deletionTime;
    private LocalDateTime deviceCreationTime;
    private double powerRatingWatts;
    private String deletionMonth; // Format: "YYYY-MM"
}
```

Smart Constructor

java

```
public DeletedDeviceEnergyRecord(Gadget device) {
    this.deviceType = device.getType();
    this.roomName = device.getRoomName();
    this.deviceModel = device.getModel();
    this.totalEnergyConsumedKWh = device.getTotalEnergyConsumedKWh();
    this.totalUsageMinutes = device.getTotalUsageMinutes();
    this.deletionTime = LocalDateTime.now();
    this.powerRatingWatts = device.getPowerRatingWatts();

    // Smart energy calculation for currently running devices
    if (device.isOn() && device.getLastOnTime() != null) {
        double currentSessionHours = device.getCurrentSessionUsageHours();
        double currentSessionEnergy = (device.getPowerRatingWatts() / 1000.0)
currentSessionHours;
        this.totalEnergyConsumedKWh += currentSessionEnergy;
        this.totalUsageMinutes += (long)(currentSessionHours  60);
    }

    // Set deletion month for easy monthly reporting
    this.deletionMonth = deletionTime.getYear() + "-" +
                         String.format("%02d", deletionTime.getMonthValue());
}
```

Smart Features:

- Current session calculation: If device is ON when deleted, includes current usage
- Energy conversion: Converts watts to kilowatt-hours for billing
- Month grouping: Stores deletion month for easy monthly reports
- Data preservation: Copies all important device information

Utility Methods

Human-Readable Time Formatting

java

```
public String getFormattedUsageTime() {
    long hours = totalUsageMinutes / 60;
    long minutes = totalUsageMinutes % 60;

    if (hours > 0) {
```

```
        return String.format("%dh %dm", hours, minutes);
    } else {
        return String.format("%dm", minutes);
    }
}
```

Mathematical Operations:
- Division (/): Converts minutes to hours
- Modulo (%): Gets remaining minutes after hour calculation
- Conditional formatting: Different display based on duration

Debug-Friendly String Representation
java

```
@Override
public String toString() {
    return String.format("DeletedDevice{%s %s in %s, Energy: %.3f kWh, Usage: %s, Deleted: %s}",
            deviceType, deviceModel, roomName, totalEnergyConsumedKWh,
            getFormattedUsageTime(), deletionTime.toString());
}
```

Example Output:
DeletedDevice{TV Samsung 55inch in Living Room, Energy: 45.230 kWh, Usage: 120h 45m, Deleted: 2024-01-15T14:30:00}

DevicePermission.java - Access Control System

File Location: `src/main/java/com/smarthome/model/DevicePermission.java`

Overview
This class implements a digital key system for smart home devices. Like giving house keys for specific rooms, it controls who can access which devices and what they can do with them.

Permission Structure
java

```
@DynamoDbBean
public class DevicePermission {
    private String memberEmail;        // Who gets permission
    private String deviceType;         // What device (TV, Light, etc.)
    private String roomName;           // Which room
    private String deviceOwnerEmail;   // Who owns the device
    private boolean canControl;        // Can turn on/off, change settings
    private boolean canView;           // Can see status, energy usage
    private LocalDateTime grantedAt;   // When permission was given
    private String grantedBy;          // Who gave the permission
}
```

Permission Levels

Permission Type	Description	Example Actions
View Only	Can see device status	Check if TV is on, view energy usage
Full Control	Can control and view	Turn TV on/off, change channels, see status
No Access	Cannot see or control	Device invisible to user

Smart Permission Matching

java

```
public boolean matchesDevice(String deviceType, String roomName, String ownerEmail) {
    return this.deviceType.equalsIgnoreCase(deviceType) &&
           this.roomName.equalsIgnoreCase(roomName) &&
           this.deviceOwnerEmail.equalsIgnoreCase(ownerEmail);
}
```

Features:
- Exact matching: Permission applies to specific device only
- Case-insensitive: Ignores capitalization differences
- Three-factor identification: Device type + room + owner

Unique Device Identification
java
```
public String getDeviceId() {
    return deviceOwnerEmail + ":" + deviceType + ":" + roomName;
}
```

Example Output: `mom@family.com:TV:Living Room`

Benefits:
- Uniqueness: No confusion between similar devices
- Traceability: Clear ownership and location
- Scalability: Works with multiple family members and devices

Real-World Usage Scenarios

Scenario 1: Family TV Access
java
```
DevicePermission familyTv = new DevicePermission(
    "john@family.com",      // John gets access
    "TV",                   // To the TV
    "Living Room",          // In living room
    "mom@family.com",       // Mom owns it
    "mom@family.com"        // Mom granted it
);
// Result: John can control the living room TV
```

Scenario 2: Guest Limited Access
java
```
DevicePermission guestLight = new DevicePermission(
    "guest@email.com",      // Guest gets access
    "Light",                // To the light
    "Kitchen",              // In kitchen
    "dad@family.com",       // Dad owns it
    "dad@family.com"        // Dad granted it
);
guestLight.setCanControl(false);  // Guest can only view, not control
```

Gadget.java - Smart Device Management

File Location: `src/main/java/com/smarthome/model/Gadget.java`

Overview
The Gadget class represents individual smart home devices like TVs, lights, air conditioners, and more. It's like a digital twin of your physical devices - it knows their current state, tracks their energy usage, and can control them remotely. This class is the heart of the smart home system.

Enums (Predefined Constants)
java
```
public enum GadgetType {
    TV, AC, FAN, ROBO_VAC_MOP
}

public enum GadgetStatus {
    ON, OFF
}
```

What are Enums?
Enums are like a list of predefined choices. Think of them as multiple-choice options that never change.

- GadgetType: Lists the main categories of devices (expandable for more types)
- GadgetStatus: Device can only be ON or OFF (like a light switch)

Benefits:
- No typos: Can't accidentally write "OOON" instead of "ON"
- Code completion: IDE suggests valid options
- Type safety: Compiler prevents invalid values

Instance Variables
java
```
private String type;                    // Device type (TV, Light, etc.)
private String model;                   // Device model (Samsung 55", etc.)
private String roomName;                // Location (Living Room, Kitchen)
private String status;                  // Current state (ON/OFF)
private double powerRatingWatts;        // How much power device uses
private LocalDateTime lastOnTime;       // When device was last turned on
private LocalDateTime lastOffTime;      // When device was last turned off
private long totalUsageMinutes;         // Total time device has been used
private double totalEnergyConsumedKWh;  // Total electricity consumed
private LocalDateTime scheduledOnTime;  // Timer: when to turn on
private LocalDateTime scheduledOffTime; // Timer: when to turn off
private boolean timerEnabled;           // Is timer functionality active
```

Variable Categories:

Category	Variables	Purpose
Identity	type, model, roomName	What and where is the device
State	status	Current on/off state
Power	powerRatingWatts	How much electricity it uses
Timing	lastOnTime, lastOffTime	Track when device was controlled
Usage Stats	totalUsageMinutes, totalEnergyConsumedKWh	Historical usage data
Scheduling	scheduledOnTime, scheduledOffTime, timerEnabled	Automated control

Constructors
Default Constructor
java
```
public Gadget() {
    this.status = GadgetStatus.OFF.name();
    this.powerRatingWatts = 0.0;
    this.totalUsageMinutes = 0L;
    this.totalEnergyConsumedKWh = 0.0;
    this.timerEnabled = false;
}
```

Purpose: Creates an empty device with safe default values.
Parameterized Constructor
java
```
public Gadget(String type, String model, String roomName) {
    this.type = type;
    this.model = model;
    this.roomName = roomName;
    this.status = GadgetStatus.OFF.name();
    this.powerRatingWatts = getDefaultPowerRating(type);  // Smart power assignment
    this.totalUsageMinutes = 0L;
    this.totalEnergyConsumedKWh = 0.0;
    this.timerEnabled = false;
}
```

Smart Features:
- Automatic power rating: Assigns realistic power consumption based on device type
- Safe defaults: All counters start at zero, device starts OFF

Device Control Methods
Turn Device On

```
java
public void turnOn() {
    if (!isOn()) {
        this.lastOnTime = LocalDateTime.now();
        updateUsageAndEnergy();
    }
    this.status = GadgetStatus.ON.name();
}
```

What happens when turning on:
1. Check current state: Only proceed if device is currently OFF
2. Record timestamp: Save exactly when device was turned on
3. Update energy usage: Calculate energy used since last off/on cycle
4. Change status: Set device to ON state

Turn Device Off
java

```
public void turnOff() {
    if (isOn()) {
        this.lastOffTime = LocalDateTime.now();
        updateUsageAndEnergy();
    }
    this.status = GadgetStatus.OFF.name();
}
```

What happens when turning off:
1. Check current state: Only proceed if device is currently ON
2. Record timestamp: Save exactly when device was turned off
3. Update energy usage: Calculate energy used during this ON session
4. Change status: Set device to OFF state

Toggle Device State
java

```
public void toggleStatus() {
    if (isOn()) {
        turnOff();
    } else {
        turnOn();
    }
}
```

Purpose: Switch device to opposite state (like a light switch button).
Smart Energy Calculation

Automatic Usage and Energy Updates
java

```
private void updateUsageAndEnergy() {
    if (lastOnTime != null && isOn()) {
        long minutesUsed = ChronoUnit.MINUTES.between(lastOnTime, LocalDateTime.now());
        totalUsageMinutes += minutesUsed;

        double hoursUsed = minutesUsed / 60.0;
        double energyUsed = (powerRatingWatts / 1000.0)  hoursUsed;
        totalEnergyConsumedKWh += energyUsed;
    }
```

```
}
```

Energy Calculation Steps:
1. Calculate time used: How many minutes since device was turned on
2. Convert to hours: Divide minutes by 60 for hour-based calculations
3. Calculate energy: Power (watts) × Time (hours) = Energy (watt-hours)
4. Convert to kilowatt-hours: Divide by 1000 for standard energy units
5. Add to total: Accumulate energy consumption over device lifetime

Formula: `Energy (kWh) = (Power in Watts ÷ 1000) × Time in Hours`

Default Power Ratings by Device Type
java

```
private static double getDefaultPowerRating(String deviceType) {
    switch (deviceType.toUpperCase()) {
        case "TV": return 150.0;
        case "AC": return 1500.0;
        case "FAN": return 75.0;
        case "LIGHT": return 60.0;
        case "SPEAKER": return 30.0;
        case "AIR_PURIFIER": return 45.0;
        case "THERMOSTAT": return 5.0;
        case "SWITCH": return 2.0;
        case "CAMERA": return 15.0;
        case "DOOR_LOCK": return 12.0;
        case "DOORBELL": return 8.0;
        case "REFRIGERATOR": return 200.0;
        case "MICROWAVE": return 1200.0;
        case "WASHING_MACHINE": return 500.0;
        case "GEYSER": return 2000.0;
        case "WATER_PURIFIER": return 25.0;
        case "VACUUM": return 1400.0;
        default: return 50.0;
    }
}
```

Power Rating Categories:

Power Level	Device Examples	Wattage Range
Very Low	Doorbell, Switch, Thermostat	2-8W
Low	Camera, Water Purifier, Speaker	12-30W
Medium	Light, Air Purifier, Fan	45-75W
High	TV, Refrigerator	150-200W
Very High	Washing Machine, Vacuum	500-1400W
Extreme	AC, Geyser, Microwave	1200-2000W

Real-Time Usage Tracking
Current Session Usage
java

```
public double getCurrentSessionUsageHours() {
    if (isOn() && lastOnTime != null) {
        long currentMinutes = ChronoUnit.MINUTES.between(lastOnTime, LocalDateTime.now());
        return currentMinutes / 60.0;
    }
    return 0.0;
}
```

Purpose: Calculate how long the device has been ON in the current session.
Live Energy Consumption
java

```
public double getCurrentTotalEnergyConsumedKWh() {
    double baseEnergy = totalEnergyConsumedKWh;
```

```
    if (isOn() && lastOnTime != null) {
        double currentSessionHours = getCurrentSessionUsageHours();
        double currentSessionEnergy = (powerRatingWatts / 1000.0)  currentSessionHours;
        baseEnergy += currentSessionEnergy;
    }
    return baseEnergy;
}
```

Smart Calculation:
- Base energy: Total energy from previous sessions
- Current session energy: Energy being consumed right now
- Live total: Base + current session = real-time energy consumption

User-Friendly Display Methods
Formatted Usage Time
java

```
public String getUsageTimeFormatted() {
    long hours = totalUsageMinutes / 60;
    long minutes = totalUsageMinutes % 60;
    return String.format("%dh %02dm", hours, minutes);
}
```

Examples:
- 150 minutes → "2h 30m"
- 45 minutes → "0h 45m"
- 1440 minutes → "24h 00m"

Device Description
java

```
@Override
public String toString() {
    return String.format("%s %s in %s - %s (%.1fW)",
                         type, model, roomName, status, powerRatingWatts);
}
```

Example Output: `TV Samsung 55inch in Living Room - ON (150.0W)`

Object Equality and Identification

Custom Equals Method
java

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;

    Gadget gadget = (Gadget) obj;
    return type.equals(gadget.type) && roomName.equals(gadget.roomName);
}
```

Equality Rules:
- Same device type AND same room = equal devices
- This prevents duplicate devices (can't have two TVs in the same room)

Hash Code for Collections

```
java
@Override
public int hashCode() {
    return (type + roomName).hashCode();
}
```

Purpose: Enables efficient storage in HashSet, HashMap, and other collections.

Real-World Usage Examples

Example 1: Living Room TV

java

```
// Create a TV
Gadget livingRoomTV = new Gadget("TV", "Samsung 55inch", "Living Room");

// Turn it on at 8 PM
livingRoomTV.turnOn();

// Watch for 2 hours, then turn off
// Energy consumed: 150W × 2h = 0.3 kWh
livingRoomTV.turnOff();

// Check usage
String usage = livingRoomTV.getUsageTimeFormatted(); // "2h 00m"
double energy = livingRoomTV.getTotalEnergyConsumedKWh(); // 0.3 kWh
```

Example 2: Kitchen Microwave

```
java
// Create a microwave
Gadget microwave = new Gadget("Microwave", "LG 1000W", "Kitchen");

// Heat food for 3 minutes
microwave.turnOn();
// ... 3 minutes later ...
microwave.turnOff();

// Energy consumed: 1200W × 0.05h = 0.06 kWh
double energy = microwave.getTotalEnergyConsumedKWh(); // 0.06 kWh
```

Example 3: Bedroom AC with Timer

```
java
// Create an AC
Gadget bedroomAC = new Gadget("AC", "Voltas 1.5 Ton", "Bedroom");

// Schedule AC to turn on at 10 PM and off at 6 AM
bedroomAC.setScheduledOnTime(LocalDateTime.of(2024, 1, 15, 22, 0));
bedroomAC.setScheduledOffTime(LocalDateTime.of(2024, 1, 16, 6, 0));
bedroomAC.setTimerEnabled(true);

// If AC runs for 8 hours: 1500W × 8h = 12 kWh
```

Smart Features Summary

1. Automatic Energy Tracking

- Real-time calculation: Energy usage updates every time device is controlled
- Accurate timing: Uses precise timestamps for duration calculation
- Live monitoring: Current session usage available instantly

2. Intelligent Power Management

- Device-specific power ratings: Each device type has realistic power consumption
- Cumulative tracking: Lifetime energy consumption across all sessions
- Cost calculation ready: Energy data in standard kWh units for billing

3. User-Friendly Interface

- Formatted displays: Time shown as "2h 30m" instead of "150 minutes"
- Clear status: Device state always clear (ON/OFF)
- Descriptive output: Full device information in readable format

4. Smart Validation

- Duplicate prevention: Can't create two identical devices in same room
- State management: Device can't be turned off if already off (and vice versa)
- Data integrity: All calculations handle edge cases safely

CalendarEventService.java - Event Management and Automation

File Location: `src/main/java/com/smarthome/service/CalendarEventService.java`

Overview

The CalendarEventService class is like a smart personal assistant that manages your calendar events and automatically controls your smart home devices based on your schedule. When you have a meeting, movie night, or bedtime, it can automatically turn on/off the right devices at the right time. This class implements the Singleton design pattern, meaning there's only one instance managing all calendar events for all users.

Package Declaration and Imports

java

```
package com.smarthome.service;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
```

Explanation:
- Package declaration: This belongs to the service package (business logic layer)
- LocalDateTime: Handles dates and times for event scheduling
- DateTimeFormatter: Formats dates for display (like "15-01-2024 14:30")
- Collections imports: ArrayList, HashMap, List, Map for storing events and data
- No database imports: This is a service class, not a model class

Singleton Pattern Implementation

java

```
public class CalendarEventService {

    private static CalendarEventService instance;
    private final Map<String, List<CalendarEvent>> userEvents;

    private CalendarEventService() {
        this.userEvents = new HashMap<>();
    }

    public static synchronized CalendarEventService getInstance() {
        if (instance == null) {
            instance = new CalendarEventService();
        }
        return instance;
    }
```

What is the Singleton Pattern?
- Only one instance: There can only be one CalendarEventService object in the entire program
- Global access: Any part of the program can access this same instance
- Shared data: All users' events are stored in the same service instance

How it works:
- `private static CalendarEventService instance;` - Stores the single instance
- `private CalendarEventService()` - Constructor is private (can't create instances directly)
- `synchronized` - Prevents multiple threads from creating multiple instances
- `if (instance == null)` - Only creates instance if it doesn't exist yet

Benefits:
- Memory efficient: Only one service object exists

- Shared state: All users' events stored together
- Consistent access: Same instance accessed from anywhere

Inner Classes (Classes Within Classes)

CalendarEvent Class

java

```
public static class CalendarEvent {
    private String eventId;
    private String title;
    private String description;
    private LocalDateTime startTime;
    private LocalDateTime endTime;
    private String eventType;
    private List<AutomationAction> automationActions;
    private boolean isRecurring;
    private String recurrencePattern;

    public CalendarEvent(String eventId, String title, String description,
                         LocalDateTime startTime, LocalDateTime endTime, String eventType) {
        this.eventId = eventId;
        this.title = title;
        this.description = description;
        this.startTime = startTime;
        this.endTime = endTime;
        this.eventType = eventType;
        this.automationActions = new ArrayList<>();
        this.isRecurring = false;
    }
}
```

What this class stores:

Variable	Type	Purpose
eventId	String	Unique identifier (like "john_202401151430")
title	String	Event name (like "Team Meeting")
description	String	Event details
startTime	LocalDateTime	When event begins
endTime	LocalDateTime	When event ends
eventType	String	Category (meeting, movie, sleep, etc.)
automationActions	List	What devices to control
isRecurring	boolean	Does event repeat?
recurrencePattern	String	How often it repeats

Constructor Features:
- Takes essential event information
- Automatically creates empty automation list
- Sets recurring to false by default
- Each event gets a unique ID

AutomationAction Class

java

```
public static class AutomationAction {
    private String deviceType;
    private String roomName;
    private String action;
    private int minutesOffset;

    public AutomationAction(String deviceType, String roomName, String action, int
minutesOffset) {
        this.deviceType = deviceType;
        this.roomName = roomName;
```

```
        this.action = action;
        this.minutesOffset = minutesOffset;
    }
}
```

What this class stores:

Variable	Type	Purpose	Example
deviceType	String	What device to control	"TV", "LIGHT", "AC"
roomName	String	Which room	"Living Room", "Kitchen"
action	String	What to do	"ON", "OFF"
minutesOffset	int	When to execute	-5 (5 min before), 0 (at start), 15 (15 min after)

Timing Examples:
- `minutesOffset = -10` → Execute 10 minutes before event starts
- `minutesOffset = 0` → Execute exactly when event starts
- `minutesOffset = 15` → Execute 15 minutes after event starts

Event Creation and Management

Creating Calendar Events

java

```
public boolean createEvent(String userEmail, String title, String description,
                           LocalDateTime startTime, LocalDateTime endTime, String eventType) {
    try {
        String eventId = generateEventId(userEmail, startTime);
        CalendarEvent event = new CalendarEvent(eventId, title, description, startTime, endTime,
eventType);

        userEvents.computeIfAbsent(userEmail, k -> new ArrayList<>()).add(event);

        addDefaultAutomationForEventType(event, eventType);

        System.out.println("[SUCCESS] Calendar event created: " + title);
        System.out.println("Event Date: " + startTime.format(DateTimeFormatter.ofPattern("dd-MM-
yyyy HH:mm")) +
                           " to " + endTime.format(DateTimeFormatter.ofPattern("HH:mm")));
        return true;

    } catch (Exception e) {
        System.out.println("[ERROR] Failed to create calendar event: " + e.getMessage());
        return false;
    }
}
```

Step-by-step process:
1. Generate unique ID: Creates ID like "john_202401151430" from email and time
2. Create event object: Uses the CalendarEvent constructor
3. Store in user's list: `computeIfAbsent()` creates list if doesn't exist, then adds event
4. Add automation: Automatically adds smart device actions based on event type
5. User feedback: Prints success message with formatted date/time
6. Error handling: Catches and reports any errors

Advanced Java Features:
- `computeIfAbsent()` - Smart map operation that creates list if key doesn't exist
- `DateTimeFormatter.ofPattern()` - Custom date formatting
- Exception handling with try-catch blocks

Smart Default Automation by Event Type

java

```
private void addDefaultAutomationForEventType(CalendarEvent event, String eventType) {
```

```
    switch (eventType.toLowerCase()) {
        case "meeting":
        case "conference":
            event.addAutomationAction(new AutomationAction("LIGHT", "Study Room", "ON", -5));
            event.addAutomationAction(new AutomationAction("AC", "Study Room", "ON", -10));
            event.addAutomationAction(new AutomationAction("LIGHT", "Study Room", "OFF", 15));
            break;

        case "movie":
        case "entertainment":
            event.addAutomationAction(new AutomationAction("TV", "Living Room", "ON", -2));
            event.addAutomationAction(new AutomationAction("LIGHT", "Living Room", "OFF", 0));
            event.addAutomationAction(new AutomationAction("AC", "Living Room", "ON", -15));
            break;

        case "sleep":
        case "bedtime":
            event.addAutomationAction(new AutomationAction("LIGHT", "Master Bedroom", "OFF",
0));
            event.addAutomationAction(new AutomationAction("TV", "Master Bedroom", "OFF", 0));
            event.addAutomationAction(new AutomationAction("AC", "Master Bedroom", "ON", -5));
            break;
    }
}
```

Automation Scenarios:

Event Type	Smart Actions	Timing Logic
Meeting/Conference	• Study room lights ON • Study room AC ON • Study room lights OFF	• 5 min before • 10 min before • 15 min after
Movie/Entertainment	• Living room TV ON • Living room lights OFF • Living room AC ON	• 2 min before • At start • 15 min before
Sleep/Bedtime	• Bedroom lights OFF • Bedroom TV OFF • Bedroom AC ON	• At bedtime • At bedtime • 5 min before
Cooking/Meal	• Kitchen lights ON • Kitchen air purifier ON • Microwave OFF	• 5 min before • At start • 30 min after
Workout/Exercise	• Living room fan ON • Speaker ON • Air purifier ON	• 5 min before • At start • At start

Why this is smart:
- Context-aware: Different actions for different activities
- Proactive: Prepares environment before you need it
- Cleanup: Automatically turns off devices after events
- Customizable: Each event type has logical device combinations

Event Display and Management

Displaying Upcoming Events

java

```
public void displayUpcomingEvents(String userEmail) {
    System.out.println("\n=== Upcoming Calendar Events ===");

    List<CalendarEvent> events = userEvents.getOrDefault(userEmail, new ArrayList<>());
    LocalDateTime now = LocalDateTime.now();

    List<CalendarEvent> upcomingEvents = events.stream()
        .filter(event -> event.getStartTime().isAfter(now))
        .sorted((e1, e2) -> e1.getStartTime().compareTo(e2.getStartTime()))
        .toList();

    if (upcomingEvents.isEmpty()) {
        System.out.println("No upcoming events scheduled.");
        return;
```

```
    }

    for (int i = 0; i < upcomingEvents.size() && i < 10; i++) {
        CalendarEvent event = upcomingEvents.get(i);
        System.out.printf("%d. %s (%s)\n", (i + 1), event.getTitle(), event.getEventType());
        System.out.printf("   [DATE] %s - %s\n",
                          event.getStartTime().format(DateTimeFormatter.ofPattern("dd-MM-yyyy
HH:mm")),
                          event.getEndTime().format(DateTimeFormatter.ofPattern("HH:mm")));

        if (!event.getAutomationActions().isEmpty()) {
            System.out.printf("   [AUTO] %d automation actions configured\n",
event.getAutomationActions().size());
        }

        if (!event.getDescription().isEmpty()) {
            System.out.printf("   [INFO] %s\n", event.getDescription());
        }
        System.out.println();
    }
}
```

Advanced Stream Processing:
1. Filter future events: `.filter(event -> event.getStartTime().isAfter(now))`
2. Sort by time: `.sorted((e1, e2) -> e1.getStartTime().compareTo(e2.getStartTime()))`
3. Convert to list: `.toList()` (modern Java feature)

Display Features:
- Limited results: Shows maximum 10 upcoming events
- Formatted output: Clean, organized display with icons [DATE], [AUTO], [INFO]
- Smart information: Only shows automation count if actions exist
- User-friendly dates: "15-01-2024 14:30" format instead of technical format

Event Automation Details
java

```
public void displayEventAutomation(String userEmail, String eventTitle) {
    List<CalendarEvent> events = userEvents.getOrDefault(userEmail, new ArrayList<>());

    CalendarEvent event = events.stream()
        .filter(e -> e.getTitle().equalsIgnoreCase(eventTitle))
        .findFirst()
        .orElse(null);

    if (event == null) {
        System.out.println("[ERROR] Event not found: " + eventTitle);
        return;
    }

    System.out.println("\n=== Event Automation Details ===");
    System.out.println("Event: " + event.getTitle() + " (" + event.getEventType() + ")");
    System.out.println("Time: " + event.getStartTime().format(DateTimeFormatter.ofPattern("dd-
MM-yyyy HH:mm")) +
                       " to " + event.getEndTime().format(DateTimeFormatter.ofPattern("HH:mm")));

    if (event.getAutomationActions().isEmpty()) {
        System.out.println("No automation actions configured for this event.");
        return;
    }

    System.out.println("\nAutomation Actions:");
    for (AutomationAction action : event.getAutomationActions()) {
        String timing = action.getMinutesOffset() == 0 ? "at event start" :
                        action.getMinutesOffset() < 0 ? (Math.abs(action.getMinutesOffset()) + "
min before") :
                        (action.getMinutesOffset() + " min after");
```

```
        System.out.printf("- %s %s in %s → %s (%s)\n",
                          action.getDeviceType(), action.getAction(), action.getRoomName(),
                          timing, action.getAction());
    }
}
```

Smart Timing Display:
- At start: `minutesOffset == 0` → "at event start"
- Before event: `minutesOffset < 0` → "5 min before"
- After event: `minutesOffset > 0` → "15 min after"
- Math.abs(): Converts negative numbers to positive for display

Example Output:
=== Event Automation Details ===
Event: Team Meeting (meeting)
Time: 15-01-2024 14:30 to 15:30

Automation Actions:
- LIGHT ON in Study Room → 5 min before
- AC ON in Study Room → 10 min before
- LIGHT OFF in Study Room → 15 min after

Real-Time Event Processing
Finding Events for Automation Triggers

java

```
public List<CalendarEvent> getEventsForAutomation(String userEmail, LocalDateTime checkTime) {
    List<CalendarEvent> events = userEvents.getOrDefault(userEmail, new ArrayList<>());
    List<CalendarEvent> triggeredEvents = new ArrayList<>();

    for (CalendarEvent event : events) {
        for (AutomationAction action : event.getAutomationActions()) {
            LocalDateTime triggerTime =
event.getStartTime().plusMinutes(action.getMinutesOffset());

            if (Math.abs(java.time.Duration.between(checkTime, triggerTime).toMinutes()) <= 1) {
                triggeredEvents.add(event);
                break;
            }
        }
    }

    return triggeredEvents;
}
```

How Automation Timing Works:
1. Calculate trigger time: `event.getStartTime().plusMinutes(action.getMinutesOffset())`
   - For -5 offset: event start time minus 5 minutes
   - For 0 offset: exactly at event start time
   - For 15 offset: event start time plus 15 minutes
2. Check if it's time: `Duration.between(checkTime, triggerTime).toMinutes()`
   - Calculates difference between current time and trigger time
   - `Math.abs()` makes the difference always positive
   - `<= 1` means within 1 minute (handles small timing variations)
3. Add to triggered list: Events ready for automation execution

Example Scenario:
Meeting starts at 14:30
- Light ON action: offset -5 → trigger at 14:25

- AC ON action: offset -10 → trigger at 14:20
- Light OFF action: offset 15 → trigger at 14:45

At 14:25, this method finds the meeting event because
it's time to turn on the study room lights.

Utility Methods
Event ID Generation

java
```
private String generateEventId(String userEmail, LocalDateTime startTime) {
    return userEmail.split("@")[0] + "_" +
startTime.format(DateTimeFormatter.ofPattern("yyyyMMddHHmm"));
}
```

ID Creation Process:
- `userEmail.split("@")[0]` → Extracts username from email (john@family.com → john)
- `startTime.format()` → Formats date as "yyyyMMddHHmm" (202401151430)
- Result: "john_202401151430" (unique for each user and time)

Benefits:
- Uniqueness: Each event has a unique identifier
- Readability: Can understand who and when from the ID
- Sortability: IDs naturally sort by date/time

Help System
java
```
public String getCalendarHelp() {
    StringBuilder help = new StringBuilder();
    help.append("\n=== Calendar Events Help ===\n");
    help.append("Available Event Types and Their Auto-Automation:\n\n");

    help.append("[1] MEETING/CONFERENCE:\n");
    help.append("   - Lights ON in Study Room (5 min before)\n");
    help.append("   - AC ON in Study Room (10 min before)\n");
    help.append("   - Lights OFF in Study Room (15 min after)\n\n");

    // ... more event types

    return help.toString();
}
```

StringBuilder Benefits:
- Efficient: Better than string concatenation with +
- Readable: Easy to build multi-line help text
- Memory friendly: Doesn't create multiple string objects

Real-World Usage Examples

Example 1: Creating a Movie Night Event
java
```
CalendarEventService service = CalendarEventService.getInstance();

// Create movie event for Friday 8 PM
LocalDateTime movieStart = LocalDateTime.of(2024, 1, 19, 20, 0);  // 8:00 PM
LocalDateTime movieEnd = LocalDateTime.of(2024, 1, 19, 22, 30);   // 10:30 PM

boolean success = service.createEvent(
```

```
    "john@family.com",
    "Friday Movie Night",
    "Watching Marvel movie with family",
    movieStart,
    movieEnd,
    "movie"
);

// Automatic actions created:
// - TV ON in Living Room (7:58 PM - 2 min before)
// - Lights OFF in Living Room (8:00 PM - at start)
// - AC ON in Living Room (7:45 PM - 15 min before)
```

Example 2: Business Meeting Setup

java

```
// Create meeting event
LocalDateTime meetingStart = LocalDateTime.of(2024, 1, 20, 14, 30);  // 2:30 PM
LocalDateTime meetingEnd = LocalDateTime.of(2024, 1, 20, 15, 30);    // 3:30 PM

service.createEvent(
    "sarah@company.com",
    "Project Review Meeting",
    "Quarterly review with team",
    meetingStart,
    meetingEnd,
    "meeting"
);

// Automatic actions:
// - Study room lights ON (2:25 PM - 5 min before)
// - Study room AC ON (2:20 PM - 10 min before)
// - Study room lights OFF (3:45 PM - 15 min after)
```

Example 3: Smart Bedtime Routine

java

```
// Create sleep event
LocalDateTime sleepStart = LocalDateTime.of(2024, 1, 20, 22, 0);   // 10:00 PM
LocalDateTime sleepEnd = LocalDateTime.of(2024, 1, 21, 6, 30);     // 6:30 AM

service.createEvent(
    "mom@family.com",
    "Bedtime",
    "Family sleep time",
    sleepStart,
    sleepEnd,
    "sleep"
);

// Automatic actions:
// - All bedroom lights OFF (10:00 PM - at bedtime)
// - Bedroom TV OFF (10:00 PM - at bedtime)
// - Bedroom AC ON (9:55 PM - 5 min before for cooling)
```

Advanced Features

Event Type Categories

The service supports 7 main event categories with smart automation:

Category	Event Types	Smart Focus
Work	meeting, conference	Study room preparation
Entertainment	movie, entertainment	Living room ambiance
Rest	sleep, bedtime	Bedroom comfort
Kitchen	cooking, meal	Kitchen preparation
Fitness	workout, exercise	Air circulation and music
Arrival	arrival, home	Welcome home setup

Departure	departure, leaving	Energy saving

Timing Precision
- Automation accuracy: ±1 minute precision
- Multiple actions: Different devices with different timing offsets
- Smart preparation: Most actions happen before events (proactive)
- Cleanup actions: Some actions happen after events (energy saving)

Memory Management
- Singleton pattern: Only one service instance
- Per-user storage: `Map<String, List<CalendarEvent>>` separates users
- Efficient lookups: HashMap provides O(1) user lookup
- Stream processing: Modern Java features for filtering and sorting

Summary for Beginners
This `CalendarEventService` class demonstrates several advanced programming concepts:
🎯 Why This Class Exists:
- Context-aware automation: Smart home responds to your schedule
- Proactive device control: Prepares environment before you need it
- Centralized event management: One service handles all users' events
- Intelligent automation: Different device actions for different activities

🧬 Design Patterns Used:
- Singleton Pattern: Only one service instance exists
- Inner Classes: CalendarEvent and AutomationAction nested inside service
- Builder Pattern: Events built step-by-step with automation actions
- Strategy Pattern: Different automation strategies for different event types

💡 Smart Features:
- Automatic device preparation: AC turns on before meetings, TV ready for movies
- Timing intelligence: Actions happen at optimal times (before, during, after)
- Multi-device orchestration: Coordinates multiple devices for each event
- Energy efficiency: Cleanup actions to turn off unused devices

📚 Programming Concepts You Learned:
- Singleton Design Pattern: Ensuring only one instance exists
- Inner/Nested Classes: Classes defined inside other classes
- Stream API: Modern Java collection processing with filter, sort, map
- StringBuilder: Efficient string building for large text
- Duration Calculations: Working with time differences and offsets
- Map Operations: `computeIfAbsent()` for smart data structure management
- Exception Handling: Try-catch blocks for error management
- Formatted Output: `printf()` and `DateTimeFormatter` for user-friendly display

🔧 Real-World Applications:
This pattern appears in:
- Smart building systems: Office lighting and climate based on meeting schedules
- Hotel automation: Room preparation based on guest check-in/check-out
- Healthcare systems: Equipment preparation based on appointment schedules
- Entertainment venues: Stage and audio setup based on event schedules

The CalendarEventService shows how modern applications integrate time-based automation, context awareness, and proactive system behavior to create intelligent, user-friendly experiences!

CustomerService.java - Authentication and Security

File Location: `src/main/java/com/smarthome/service/CustomerService.java`

Overview

The CustomerService class is like a digital security guard for your smart home system. It handles all the critical security functions: registering new users, logging them in safely, protecting against hackers, and managing passwords. This class implements enterprise-grade security features including BCrypt password hashing, progressive account lockout, and brute force attack protection. Think of it as the fortress wall that protects all your smart home data.

Package Declaration and Imports

java

```
package com.smarthome.service;

import com.smarthome.model.Customer;
import com.smarthome.util.DynamoDBConfig;
import org.mindrot.jbcrypt.BCrypt;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbEnhancedClient;
import software.amazon.awssdk.enhanced.dynamodb.DynamoDbTable;
import software.amazon.awssdk.enhanced.dynamodb.Key;
import software.amazon.awssdk.enhanced.dynamodb.TableSchema;
import software.amazon.awssdk.services.dynamodb.model.ResourceNotFoundException;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
```

Explanation:
- Service package: This belongs to the service layer (business logic)
- Customer model: Imports the Customer class we studied earlier
- BCrypt import: Industry-standard password hashing library
- DynamoDB imports: Database connectivity and operations
- Time handling: LocalDateTime for security timing
- Collections: Arrays, HashMap, List for data management

Key Security Import:
- `org.mindrot.jbcrypt.BCrypt` - This is a professional password hashing library used by banks and major companies

Class Structure and Security Constants

java

```
public class CustomerService {

    private final DynamoDbTable<Customer> customerTable;
    private final boolean isDemoMode;
    private final Map<String, Customer> demoCustomers;

    private static final List<String> COMMON_PASSWORDS = Arrays.asList(
        "password", "123456", "password123", "admin", "qwerty", "abc123",
        "123456789", "welcome", "monkey", "1234567890", "dragon", "letmein",
        "india123", "mumbai", "delhi", "bangalore", "chennai", "kolkata",
        "password1", "admin123", "root", "toor", "pass", "test", "guest",
        "user", "demo", "sample", "temp", "change", "changeme", "default",
        "india", "bharat", "hindustan", "cricket", "bollywood", "iloveyou"
    );
```

Instance Variables:

Variable	Type	Purpose

customerTable	DynamoDbTable<Customer>	Database connection for customer data
isDemoMode	boolean	Whether running in demo mode (no database)
demoCustomers	Map<String, Customer>	In-memory storage for demo mode

Security Feature - Common Passwords Blacklist:
- 36 common passwords blocked to prevent weak security
- Regional awareness: Includes Indian city names and cultural terms
- Industry standard: Prevents passwords like "admin", "password123"
- Smart prevention: Stops users from choosing easily guessed passwords

Constructor and Database Setup
java

```
public CustomerService() {
    DynamoDbEnhancedClient enhancedClient = DynamoDBConfig.getEnhancedClient();

    if (enhancedClient != null) {
        this.customerTable = enhancedClient.table("customers",
TableSchema.fromBean(Customer.class));
        this.isDemoMode = false;
        this.demoCustomers = null;
        createTableIfNotExists();
    } else {
        System.out.println("🎮 Running in DEMO MODE - data won't persist between sessions");
        this.customerTable = null;
        this.isDemoMode = true;
        this.demoCustomers = new HashMap<>();
    }
}
```

Smart Initialization:
1. Try database connection: Attempts to connect to DynamoDB
2. Production mode: If database available, use real storage
3. Demo mode fallback: If no database, use in-memory storage
4. Table creation: Automatically creates database table if needed
5. User feedback: Clear messages about which mode is running

Benefits:
- Flexibility: Works with or without database
- Development friendly: Demo mode for testing
- Production ready: Full database integration when available

Database Table Management

java

```
private void createTableIfNotExists() {
    if (!isDemoMode) {
        try {
            customerTable.describeTable();
            System.out.println("📋 DynamoDB table 'customers' already exists");
        } catch (ResourceNotFoundException e) {
            System.out.println("🔨 Creating DynamoDB table 'customers'...");
            customerTable.createTable();
            System.out.println("✅ Successfully created 'customers' table in DynamoDB");
        } catch (Exception e) {
            System.err.println("❌ Error checking/creating table: " + e.getMessage());
            throw e;
        }
    }
}
```

Automatic Database Setup:
1. Check if table exists: `describeTable()` throws exception if not found
2. Create if missing: `createTable()` creates the table automatically
3. Error handling: Catches and reports any database issues
4. User feedback: Emojis and clear messages for each step

User Registration System
Customer Registration

java
```
public boolean registerCustomer(String fullName, String email, String password) {
    try {
        email = email.trim().toLowerCase();

        if (findCustomerByEmail(email) != null) {
            return false;
        }

        String hashedPassword = BCrypt.hashpw(password, BCrypt.gensalt());
        Customer customer = new Customer(email, fullName.trim(), hashedPassword);

        if (isDemoMode) {
            demoCustomers.put(email, customer);
            System.out.println("💾 Customer registered in DEMO mode (data will not persist)");
        } else {
            customerTable.putItem(customer);
        }
        return true;

    } catch (Exception e) {
        System.err.println("Error registering customer: " + e.getMessage());
        return false;
    }
}
```

Registration Process:
1. Email normalization: `trim().toLowerCase()` ensures consistent format
2. Duplicate check: Prevents registering same email twice
3. Password hashing: `BCrypt.hashpw()` creates secure password hash
4. Customer creation: Uses Customer constructor with hashed password
5. Storage: Saves to database or demo memory
6. Result: Returns true/false for success/failure

Security Features:
- BCrypt hashing: Industry-standard password security
- Salt generation: `BCrypt.gensalt()` adds unique salt for each password
- No plain text: Passwords never stored in readable form
- Duplicate prevention: Can't register same email twice

Authentication System
Secure Customer Login
java
```
public Customer authenticateCustomer(String email, String password) {
    try {
        email = email.trim().toLowerCase();
        Customer customer = findCustomerByEmail(email);

        if (customer == null) {
            System.out.println("[ERROR] Email not registered. Please create an account first.");
            // Add rate limiting even for non-existent accounts to prevent enumeration
            addSecurityDelay(1);
            return null;
```

```
        }

        if (customer.isAccountLocked()) {
            LocalDateTime lockUntil = customer.getAccountLockedUntil();
            System.out.println("[LOCKED] Account is locked until: " +
                lockUntil.toString().replace("T", " ") +
                ". Please try again later.");
            return null;
        }

        if (password != null && BCrypt.checkpw(password, customer.getPassword())) {
            if (customer.getFailedLoginAttempts() > 0) {
                customer.resetFailedAttempts();
                updateCustomer(customer);
                System.out.println("[SUCCESS] Login successful! Previous failed attempts have
been cleared.");
            } else {
                System.out.println("[SUCCESS] Login successful! Welcome back, " +
customer.getFullName() + "!");
            }
            return customer;
        } else {
            // Add progressive delay based on failed attempts before handling failed login
            addSecurityDelay(customer.getFailedLoginAttempts() + 1);
            handleFailedLogin(customer);
            return null;
        }

    } catch (Exception e) {
        System.err.println("Error authenticating customer: " + e.getMessage());
        return null;
    }
}
```

Multi-Layer Security Process:

1. Account Existence Check
- Email lookup: Find customer by email
- Anti-enumeration: Even non-existent accounts get security delay
- Privacy protection: Don't reveal which emails exist

2. Account Lock Check
- Lock status: `isAccountLocked()` checks if account is locked
- Lock display: Shows exactly when account will unlock
- Time formatting: Converts "T" to space for readability

3. Password Verification
- BCrypt check: `BCrypt.checkpw()` verifies password against hash
- Success handling: Clears failed attempts on successful login
- Failure handling: Adds security delay and updates fail count

4. Progressive Security
- Increasing delays: Each failed attempt adds longer delay
- Attack prevention: Makes brute force attacks impractical
- Smart logging: Records security events for monitoring

Advanced Security Features
Progressive Account Lockout
java

```
private int calculateLockoutMinutes(int failedAttempts) {
    // Enhanced security: More aggressive lockout policy
    if (failedAttempts >= 10) {
        return 1440; // 24 hours - severe security violation
```

```
    } else if (failedAttempts >= 7) {
        return 240;  // 4 hours - repeated violations
    } else if (failedAttempts >= 5) {
        return 60;   // 1 hour - persistent attempts
    } else if (failedAttempts >= 3) {
        return 15;   // 15 minutes - suspicious activity
    } else if (failedAttempts >= 2) {
        return 5;    // 5 minutes - first warning
    }
    return 0;
}
```

Escalating Security Response:

Failed Attempts	Lockout Duration	Security Level
1 attempt	No lock	Normal activity
2 attempts	5 minutes	First warning
3 attempts	15 minutes	Suspicious activity
5 attempts	1 hour	Persistent attempts
7 attempts	4 hours	Repeated violations
10+ attempts	24 hours	Severe security violation

Why This Works:
- Escalation: Each attempt makes lockout longer
- Deterrent effect: Attackers face exponentially increasing delays
- Legitimate users: Short initial delays for typos
- Serious threats: Long delays for persistent attacks

Failed Login Handler
java

```
private void handleFailedLogin(Customer customer) {
    customer.incrementFailedAttempts();
    int attempts = customer.getFailedLoginAttempts();

    // Enhanced security logging
    System.out.println("[SECURITY] Failed login attempt " + attempts + " for account: " +
                       customer.getEmail() + " at " +
java.time.LocalDateTime.now().toString().replace("T", " "));

    int lockoutMinutes = calculateLockoutMinutes(attempts);
    if (lockoutMinutes > 0) {
        customer.lockAccount(lockoutMinutes);

        if (lockoutMinutes >= 1440) {
            System.out.println("[SECURITY ALERT] Account PERMANENTLY LOCKED for 24 hours due to
" +
                               attempts + " failed attempts. Contact administrator if
legitimate.");
        } else if (lockoutMinutes >= 240) {
            System.out.println("[HIGH SECURITY RISK] Account locked for " + (lockoutMinutes/60)
+ " hours due to " +
                               attempts + " repeated failed attempts.");
        } else {
            System.out.println("[LOCKED] Account locked for " + lockoutMinutes + " minutes due
to " +
                               attempts + " failed login attempts.");
        }
    } else {
        // First attempt - give warning
        System.out.println("[WARNING] Invalid password. This is attempt " + attempts + " of 2
allowed before lockout.");
        System.out.println("[SECURITY] Account will be locked after 2 failed attempts for
security.");
    }

    updateCustomer(customer);
```

```
}
```

Comprehensive Failure Handling:
1. Increment counter: `incrementFailedAttempts()` tracks attempts
2. Security logging: Records timestamp and email for audit
3. Calculate lockout: Determines appropriate lockout duration
4. Lock account: Sets lockout end time in customer record
5. User feedback: Clear messages about security status
6. Database update: Saves changes to persistent storage

Smart User Communication:
- Warning messages: Clear explanation of security policy
- Escalation alerts: Different messages for severity levels
- Time information: Exact lockout duration communicated
- Professional tone: Security-focused but user-friendly

Progressive Security Delays
java

```
private void addSecurityDelay(int failedAttempts) {
    try {
        int delaySeconds;
        switch (failedAttempts) {
            case 1:
                delaySeconds = 1;  // 1 second - first failed attempt
                break;
            case 2:
                delaySeconds = 3;  // 3 seconds - second attempt
                break;
            case 3:
                delaySeconds = 5;  // 5 seconds - getting suspicious
                break;
            case 4:
                delaySeconds = 10; // 10 seconds - definite attack pattern
                break;
            default:
                delaySeconds = 15; // 15 seconds - severe attack pattern
                break;
        }

        if (delaySeconds > 1) {
            System.out.println("[SECURITY] Implementing " + delaySeconds + " second delay due to
failed attempts...");
        }

        Thread.sleep(delaySeconds  1000L);

    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.err.println("[SECURITY] Security delay interrupted");
    }
}
```

Progressive Delay Strategy:

Attempt	Delay	Purpose
1st	1 second	Minimal impact on typos
2nd	3 seconds	Slight deterrent
3rd	5 seconds	Clear suspicious activity
4th	10 seconds	Definite attack pattern
5th+	15 seconds	Severe attack pattern

Security Benefits:
- Brute force prevention: Makes automated attacks slow and impractical

- Immediate response: Doesn't wait for account lock to activate
- Thread safety: Properly handles interruption
- User awareness: Clear messages about security delays

Input Validation System

Email Validation

java

```
public boolean isValidEmail(String email) {
    if (email == null || email.trim().isEmpty()) {
        return false;
    }
    email = email.trim().toLowerCase();
    return email.matches("^[a-z0-9+_.-]+@[a-z0-9.-]+\\.[a-z]{2,}$");
}
```

Email Validation Rules:
- Null safety: Checks for null or empty strings
- Normalization: Converts to lowercase
- Regex pattern: `^[a-z0-9+_.-]+@[a-z0-9.-]+\\.[a-z]{2,}$`

Regex Breakdown:
- `^` - Start of string
- `[a-z0-9+_.-]+` - Username part (letters, numbers, special chars)
- `@` - Required @ symbol
- `[a-z0-9.-]+` - Domain name part
- `\\.` - Required dot (escaped)
- `[a-z]{2,}` - Domain extension (at least 2 letters)
- `$` - End of string

Valid Examples: john@gmail.com, user.name@company.co.in
Invalid Examples: @gmail.com, john@, john.gmail.com

Advanced Password Validation

java

```
public boolean isValidPassword(String password) {
    if (password == null || password.trim().isEmpty()) {
        return false;
    }

    if (password.length() < 8) {
        return false;
    }

    if (password.length() > 128) {
        return false;
    }

    if (!password.matches(".[A-Z]")) {
        return false;
    }

    if (!password.matches(".[a-z].")) {
        return false;
    }

    if (!password.matches(".[0-9].")) {
        return false;
    }

    if (!password.matches(".[!@$%^&()_+\\-=\\[\\]{}|;':\",./<>?].")) {
        return false;
```

```
    }

    if (COMMON_PASSWORDS.contains(password.toLowerCase())) {
        return false;
    }

    if (password.matches("(.)\\1{2,}")) {
        return false;
    }

    return true;
}
```

Password Security Requirements:

Rule	Pattern	Purpose
Length	8-128 characters	Adequate strength, reasonable limit
Uppercase	`.[A-Z].`	At least one capital letter
Lowercase	`.[a-z].`	At least one small letter
Numbers	`.[0-9].`	At least one digit
Special chars	`.[!@$%^&()_+...].`	At least one symbol
Not common	Not in COMMON_PASSWORDS list	Prevents weak passwords
No repeating	`(.)\\1{2,}`	Prevents "aaa" patterns

Security Benefits:
- Complexity enforcement: Multiple character types required
- Common password prevention: Blocks 36 common passwords
- Pattern prevention: Stops simple repeating characters
- Reasonable limits: Not too short (weak) or too long (impractical)

Password Requirements Display

java

```
public String getPasswordRequirements() {
    return "Password must be 8-128 characters long and contain:\n" +
           "- At least one uppercase letter (A-Z)\n" +
           "- At least one lowercase letter (a-z)\n" +
           "- At least one number (0-9)\n" +
           "- At least one special character (!@$%^&)\n" +
           "- Cannot be a common password\n" +
           "- Cannot have more than 2 repeating characters";
}
```

User-Friendly Security:
- Clear requirements: Exactly what users need to do
- Specific examples: Shows character types needed
- Helpful format: Easy to read bullet points
- Complete list: All rules explained clearly

Password Management System

Password Reset Functionality

java

```
public boolean resetPassword(String email, String newPassword) {
    try {
        Customer customer = findCustomerByEmail(email);
        if (customer == null) {
            System.out.println("[ERROR] Account not found.");
            return false;
        }

        if (!isValidPassword(newPassword)) {
            System.out.println("[ERROR] New password does not meet security requirements:");
            System.out.println(getPasswordRequirements());
            return false;
        }
```

```
        String hashedPassword = BCrypt.hashpw(newPassword, BCrypt.gensalt());
        customer.setPassword(hashedPassword);

        customer.resetFailedAttempts();

            boolean updated = updateCustomer(customer);
        if (updated) {
            System.out.println("[SUCCESS] Password reset successful! You can now login with your
new password.");
        } else {
            System.out.println("[ERROR] Failed to update password. Please try again.");
        }

        return updated;

    } catch (Exception e) {
        System.err.println("Error resetting password: " + e.getMessage());
        return false;
    }
}
```

Password Reset Process:
1. Account verification: Ensures customer exists
2. Password validation: Checks new password meets requirements
3. Secure hashing: Creates new BCrypt hash with fresh salt
4. Clear attempts: Resets failed login counter
5. Database update: Saves changes to storage
6. User feedback: Clear success/failure messages

Security Features:
- Fresh hash: New salt generated for new password
- Requirements check: Same strict validation as registration
- Account unlock: Clears any existing lockouts
- Complete update: Saves all security changes

Database Operations
Customer Lookup
java

```
public Customer findCustomerByEmail(String email) {
    try {
        if (email == null || email.trim().isEmpty()) {
            return null;
        }

        email = email.trim().toLowerCase();

        Customer customer = null;
        if (isDemoMode) {
            customer = demoCustomers.get(email);
        } else {
            Key key = Key.builder().partitionValue(email).build();
            customer = customerTable.getItem(key);
        }

        if (customer != null && customer.getGadgets() != null) {
            for (com.smarthome.model.Gadget gadget : customer.getGadgets()) {
                gadget.ensurePowerRating();
            }
        }

        return customer;

    } catch (Exception e) {
        System.err.println("Error finding customer: " + e.getMessage());
```

```
        return null;
    }
}
```

Smart Customer Retrieval:
1. Input validation: Checks for null/empty email
2. Normalization: Consistent email format
3. Mode-aware lookup: Different logic for demo vs. production
4. DynamoDB key: Uses email as partition key
5. Gadget maintenance: Ensures power ratings are set
6. Error handling: Graceful failure with logging

Customer Updates
java

```
public boolean updateCustomer(Customer customer) {
    try {
        if (isDemoMode) {
            demoCustomers.put(customer.getEmail().toLowerCase(), customer);
        } else {
            customerTable.updateItem(customer);
        }
        return true;

    } catch (Exception e) {
        System.err.println("Error updating customer: " + e.getMessage());
        return false;
    }
}
```

Update Strategy:
- Mode flexibility: Works in both demo and production modes
- Key consistency: Uses lowercase email as key
- DynamoDB integration: Uses enhanced client for updates
- Result feedback: Returns success/failure status

Real-World Usage Examples

Example 1: New User Registration

```
java
CustomerService service = new CustomerService();

// Attempt to register new user
String fullName = "John Smith";
String email = "john.smith@gmail.com";
String password = "MySecure123!";

// Check if email is available
if (!service.isEmailAvailable(email)) {
    System.out.println("Email already registered!");
    return;
}

// Validate password strength
if (!service.isValidPassword(password)) {
    System.out.println("Password doesn't meet requirements:");
    System.out.println(service.getPasswordRequirements());
    return;
}

// Register the customer
boolean success = service.registerCustomer(fullName, email, password);
if (success) {
    System.out.println("Registration successful!");
```

```
} else {
    System.out.println("Registration failed!");
}
```

Registration Flow:
1. Email availability: Check if email is already used
2. Password validation: Ensure password meets security requirements
3. Registration attempt: Create new customer account
4. Result handling: Appropriate feedback to user

Example 2: Secure Login Process
java

```
// Attempt to login
String email = "john.smith@gmail.com";
String password = "MySecure123!";

Customer loggedInCustomer = service.authenticateCustomer(email, password);

if (loggedInCustomer != null) {
    System.out.println("Welcome, " + loggedInCustomer.getFullName());
    // Proceed with smart home dashboard
} else {
    System.out.println("Login failed. Check your credentials.");
    // Show login form again
}
```

Authentication Results:
- Success: Returns Customer object for session management
- Failure: Returns null and shows appropriate error message
- Security: Automatic delays and lockouts handled internally

Example 3: Password Reset Scenario

```
java
// User forgot password
String email = "john.smith@gmail.com";
String newPassword = "NewSecure456!";

// Initiate password reset
if (service.initiatePasswordReset(email)) {
    System.out.println("Account found! You can reset your password.");

    // Reset password
    boolean resetSuccess = service.resetPassword(email, newPassword);
    if (resetSuccess) {
        System.out.println("Password reset successful! You can now login.");
    } else {
        System.out.println("Password reset failed. Check requirements.");
    }
} else {
    System.out.println("No account found with this email.");
}
```

Reset Process:
1. Account verification: Confirm email exists
2. Password validation: New password must meet requirements
3. Secure update: Hash and store new password
4. Account unlock: Clear any existing security locks

Enterprise Security Features Summary

🔐 Authentication Security
- BCrypt hashing: Industry-standard password protection

- Unique salts: Each password gets unique salt
- Progressive delays: Increasing delays for failed attempts
- Account lockout: Automatic protection against brute force

🛡️ Input Validation
- Email validation: Regex pattern matching
- Password complexity: 7 different security requirements
- Common password prevention: Blocks 36 weak passwords
- Name validation: Ensures proper names only

📊 Attack Prevention
- Anti-enumeration: Delays even for non-existent accounts
- Progressive escalation: 1 second to 24 hour lockouts
- Security logging: Detailed failure tracking
- Thread safety: Proper interrupt handling

💾 Data Management
- Dual mode operation: Demo and production modes
- Automatic table creation: Sets up database automatically
- Consistent key usage: Email-based primary keys
- Error resilience: Graceful failure handling

Summary for Beginners

This `CustomerService` class demonstrates professional-grade security implementation:

🎯 Why This Class Exists:
- User authentication: Secure login and registration
- Password security: Industry-standard protection
- Attack prevention: Stops hackers and automated attacks
- Data protection: Safely manages sensitive user information

⚗️ Security Patterns Used:
- BCrypt Hashing: Never store plain text passwords
- Progressive Security: Escalating response to threats
- Input Validation: Prevent malicious data entry
- Rate Limiting: Slow down attackers automatically

💡 Advanced Features:
- Multi-mode operation: Works with or without database
- Automatic lockout: Protects accounts from brute force
- Smart feedback: User-friendly security messages
- Enterprise ready: Security practices used by major companies

📚 Programming Concepts You Learned:
- Security Libraries: Using BCrypt for password hashing
- Regex Patterns: Advanced string matching for validation
- Exception Handling: Comprehensive error management
- Thread Management: `Thread.sleep()` and interrupt handling
- Map Operations: HashMap for efficient data storage
- Conditional Logic: Complex security decision trees
- String Manipulation: Email normalization and formatting
- Time Calculations: Security delays and lockout timing

🔧 Real-World Applications:
This security pattern appears in:
- Banking systems: Account security and fraud prevention
- Social media platforms: User authentication and protection
- E-commerce sites: Customer account management
- Healthcare systems: Patient data protection
- Government portals: Citizen service security

The CustomerService shows how modern applications implement defense-in-depth security, user-friendly authentication, and enterprise-grade protection against real-world cyber threats!

DeviceHealthService.java - Health Monitoring and Maintenance

File Location: `src/main/java/com/smarthome/service/DeviceHealthService.java`

Overview

The DeviceHealthService class is like a digital doctor for your smart home devices. It continuously monitors the health of all your devices, diagnoses potential problems, predicts when maintenance is needed, and provides actionable recommendations. Think of it as a comprehensive health monitoring system that helps prevent device failures, optimize energy efficiency, and extend device lifespan through predictive maintenance.

Package Declaration and Imports

java

```
package com.smarthome.service;

import com.smarthome.model.Customer;
import com.smarthome.model.Gadget;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;
import java.util.;
```

Explanation:
- Service package: Business logic layer for health monitoring
- Model imports: Customer and Gadget classes for device analysis
- Time handling: LocalDateTime for timestamps, ChronoUnit for time calculations
- Collections: ArrayList, List, Map for managing health data
- Random: For simulating real-world device variations

Singleton Pattern Implementation

java

```
public class DeviceHealthService {

    private static DeviceHealthService instance;
    private final Random random;

    private DeviceHealthService() {
        this.random = new Random();
    }

    public static synchronized DeviceHealthService getInstance() {
        if (instance == null) {
            instance = new DeviceHealthService();
        }
        return instance;
    }
```

Why Singleton for Health Service:

- Single monitoring system: One centralized health monitor for all devices
- Consistent analysis: Same algorithms and thresholds across all devices
- Memory efficient: Shared instance across the entire application
- Random seed consistency: Same Random instance for consistent simulations

Inner Classes for Health Data
DeviceHealth Class

```
java
public static class DeviceHealth {
    private String deviceType;
    private String model;
    private String roomName;
    private int healthScore;
    private String healthStatus;
    private List<String> issues;
    private List<String> recommendations;
    private LocalDateTime lastCheckTime;
    private int estimatedLifespanMonths;
    private double energyEfficiencyScore;

    public DeviceHealth(String deviceType, String model, String roomName) {
        this.deviceType = deviceType;
        this.model = model;
        this.roomName = roomName;
        this.issues = new ArrayList<>();
        this.recommendations = new ArrayList<>();
        this.lastCheckTime = LocalDateTime.now();
    }
}
```

Health Data Structure:

Field	Type	Purpose
deviceType	String	What kind of device (TV, AC, etc.)
model	String	Specific model information
roomName	String	Device location
healthScore	int	Overall health rating (0-100)
healthStatus	String	EXCELLENT, GOOD, WARNING, CRITICAL
issues	List<String>	Current problems detected
recommendations	List<String>	Suggested actions
lastCheckTime	LocalDateTime	When health was last checked
estimatedLifespanMonths	int	Expected remaining life
energyEfficiencyScore	double	Energy efficiency rating (0-100)

SystemHealthReport Class
java

```
public static class SystemHealthReport {
    private int totalDevices;
    private int healthyDevices;
    private int warningDevices;
    private int criticalDevices;
    private List<DeviceHealth> deviceHealthList;
    private double overallSystemHealth;
    private List<String> systemRecommendations;

    public SystemHealthReport() {
        this.deviceHealthList = new ArrayList<>();
        this.systemRecommendations = new ArrayList<>();
    }
}
```

System-Wide Monitoring:

- Device counts: Tracks healthy, warning, and critical devices
- Overall health: Average health score across all devices
- Comprehensive list: Individual health data for each device
- System recommendations: Actions for overall system improvement

Health Analysis Engine

Comprehensive Device Health Analysis

java

```
private DeviceHealth analyzeDeviceHealth(Gadget device) {
    DeviceHealth health = new DeviceHealth(device.getType(), device.getModel(),
device.getRoomName());

    // Simulate device health analysis based on usage patterns and device type
    int baseHealth = 85 + random.nextInt(15); // Base health 85-100

    // Adjust based on usage time
    long totalMinutes = device.getTotalUsageMinutes();
    if (totalMinutes > 10080) { // More than 1 week of usage
        baseHealth -= 5;
        health.addIssue("High usage detected");
        health.addRecommendation("Consider reducing usage or scheduling maintenance");
    }
```

Smart Health Calculation:

1. Base health: Starts with 85-100% (simulates manufacturing quality variation)
2. Usage analysis: Penalizes excessive usage (>1 week total runtime)
3. Device-specific factors: Different analysis for different device types
4. Real-time assessment: Considers current device state
5. Efficiency scoring: Evaluates energy consumption vs. expected values

Device-Specific Health Rules

java

```
// Adjust based on device type
switch (device.getType().toUpperCase()) {
    case "AC":
        if (baseHealth > 70) {
            health.addRecommendation("Clean AC filters monthly for optimal performance");
        }
        health.setEstimatedLifespanMonths(120); // 10 years
        break;
    case "REFRIGERATOR":
        if (device.getPowerRatingWatts() > 250) {
            health.addIssue("High power consumption detected");
            baseHealth -= 10;
        }
        health.setEstimatedLifespanMonths(180); // 15 years
        break;
    case "GEYSER":
        health.addRecommendation("Check for water leakage and heating efficiency");
        health.setEstimatedLifespanMonths(96); // 8 years
        break;
}
```

Device Lifespan Expectations:

Device Type	Expected Lifespan	Common Issues
AC	10 years (120 months)	Filter clogging, refrigerant leaks
Refrigerator	15 years (180 months)	High power consumption, seal issues
Geyser	8 years (96 months)	Water leakage, heating efficiency
Washing Machine	12 years (144 months)	Mechanical wear, water damage
TV	7 years (84 months)	Overheating, display degradation
Default	5 years (60 months)	General electronics lifespan

Real-World Issue Simulation

java

```
// Simulate potential issues
if (random.nextInt(100) < 15) { // 15% chance of connectivity issue
    health.addIssue("Intermittent connectivity detected");
    health.addRecommendation("Check Wi-Fi signal strength in " + device.getRoomName());
    baseHealth -= 10;
}

if (random.nextInt(100) < 10) { // 10% chance of firmware issue
    health.addIssue("Firmware update available");
    health.addRecommendation("Update device firmware for security and performance");
    baseHealth -= 5;
}

if (device.isOn()) {
    double currentSessionHours = device.getCurrentSessionUsageHours();
    if (currentSessionHours > 8) { // Running for more than 8 hours
        health.addIssue("Extended continuous operation");
        health.addRecommendation("Consider giving device a break to prevent overheating");
        baseHealth -= 8;
    }
}
```

Realistic Problem Detection:

- Connectivity issues (15% chance): Common IoT device problem
- Firmware problems (10% chance): Security and performance updates needed
- Overheating protection: Warns about extended continuous operation (>8 hours)
- Health impact: Each issue reduces overall health score appropriately

Energy Efficiency Analysis

java

```
// Set energy efficiency score (ensure <= 100%)
double expectedPower = getExpectedPowerRating(device.getType());
double efficiencyScore = Math.max(0, 100 - ((device.getPowerRatingWatts() - expectedPower) /
expectedPower  100));
health.setEnergyEfficiencyScore(Math.min(100, efficiencyScore));

// Adjust health based on energy efficiency
if (health.getEnergyEfficiencyScore() < 70) {
    health.addIssue("Below average energy efficiency");
    health.addRecommendation("Consider upgrading to energy-efficient model");
    baseHealth -= 5;
}
```

Efficiency Calculation Process:

1. Get expected power: Standard power rating for device type
2. Calculate variance: How much actual power deviates from expected
3. Score calculation: `100 - (deviation percentage)`
4. Bounds checking: Ensure score stays between 0-100
5. Health impact: Poor efficiency affects overall health score

Health Status Classification

java

```
// Set health status
if (health.getHealthScore() >= 80) {
    health.setHealthStatus("EXCELLENT");
} else if (health.getHealthScore() >= 60) {
    health.setHealthStatus("GOOD");
} else if (health.getHealthScore() >= 40) {
    health.setHealthStatus("WARNING");
} else {
    health.setHealthStatus("CRITICAL");
}
```

Health Categories:

Score Range	Status	Meaning	Action Required
80-100	EXCELLENT	Perfect condition	Routine maintenance only
60-79	GOOD	Minor issues	Preventive maintenance
40-59	WARNING	Significant problems	Scheduled maintenance needed
0-39	CRITICAL	Major failures	Immediate attention required

Advanced Diagnostic Features
Detailed Problem Analysis for Low Health
java

```
private void addLowHealthReasons(DeviceHealth health, Gadget device) {
    int healthScore = health.getHealthScore();

    // Specific reasons based on health score ranges
    if (healthScore < 40) {
        health.addIssue("[CRITICAL] Health score below 40% - Multiple system failures
detected");
        health.addRecommendation("[URGENT] Schedule immediate professional inspection");
        health.addRecommendation("[ACTION] Consider device replacement if repair costs exceed
60% of new device");
    } else if (healthScore < 50) {
        health.addIssue("[SEVERE] Health score below 50% - Significant performance
degradation");
        health.addRecommendation("[PRIORITY] Professional maintenance required within 1 week");
        health.addRecommendation("[CHECK] Verify power supply and connections");
    } else if (healthScore < 60) {
        health.addIssue("[WARNING] Health score below 60% - Performance issues detected");
        health.addRecommendation("[SCHEDULE] Maintenance check within 2 weeks recommended");
    }
}
```

Escalating Diagnostic Response:
- Critical (<40%): Multiple system failures, immediate professional help
- Severe (<50%): Significant degradation, priority maintenance within 1 week
- Warning (<60%): Performance issues, maintenance within 2 weeks

Device-Specific Low Health Analysis

```
java
// Device-specific low health reasons
String deviceType = device.getType().toUpperCase();
switch (deviceType) {
    case "AC":
        if (healthScore < 60) {
            health.addIssue("AC performance degraded - possible refrigerant leak or dirty
filters");
            health.addRecommendation("Check and clean/replace filters, inspect for refrigerant
leaks");
        }
        break;
    case "REFRIGERATOR":
        if (healthScore < 60) {
            health.addIssue("Refrigerator efficiency compromised - possible seal/coil issues");
            health.addRecommendation("Check door seals, clean condenser coils, verify
temperature settings");
        }
        break;
}
```

Specialized Diagnostics:
- AC problems: Refrigerant leaks, dirty filters, cooling efficiency
- Refrigerator issues: Door seals, condenser coils, temperature control

- Geyser problems: Sediment buildup, heating element wear
- Washing machine: Mechanical wear, water damage, drum balance

System-Wide Health Reporting

Comprehensive Health Report Generation

```
java
public SystemHealthReport generateHealthReport(Customer user) {
    SystemHealthReport report = new SystemHealthReport();
    List<Gadget> devices = user.getGadgets();

    if (devices == null || devices.isEmpty()) {
        return report;
    }

    int totalHealthScore = 0;
    int healthyCount = 0, warningCount = 0, criticalCount = 0;

    for (Gadget device : devices) {
        DeviceHealth health = analyzeDeviceHealth(device);
        report.addDeviceHealth(health);

        totalHealthScore += health.getHealthScore();

        if (health.getHealthScore() >= 80) {
            healthyCount++;
        } else if (health.getHealthScore() >= 60) {
            warningCount++;
        } else {
            criticalCount++;
        }
    }

  report.setTotalDevices(devices.size());
  report.setHealthyDevices(healthyCount);
  report.setWarningDevices(warningCount);
  report.setCriticalDevices(criticalCount);

  if (devices.size() > 0) {
     report.setOverallSystemHealth(totalHealthScore / (double) devices.size());
  }

  generateSystemRecommendations(report);
  return report;
}
```

Report Generation Process:

1. Individual analysis: Check health of each device
2. Categorization: Sort devices into healthy/warning/critical groups
3. Statistical analysis: Calculate overall system health average
4. System recommendations: Generate comprehensive maintenance advice
5. Complete report: Package all data into structured report object

Smart System Recommendations

java

```
private void generateSystemRecommendations(SystemHealthReport report) {
    if (report.getCriticalDevices() > 0) {
        report.addSystemRecommendation("[URGENT] " + report.getCriticalDevices() + " device(s)
need immediate attention");
    }

    if (report.getWarningDevices() > 0) {
```

```
        report.addSystemRecommendation("[WARNING] " + report.getWarningDevices() + " device(s)
require maintenance soon");
    }

    if (report.getOverallSystemHealth() > 90) {
        report.addSystemRecommendation("[EXCELLENT] System health! Keep up the good
maintenance");
    } else if (report.getOverallSystemHealth() > 75) {
        report.addSystemRecommendation("[GOOD] System health overall, minor optimizations
recommended");
    } else if (report.getOverallSystemHealth() > 60) {
        report.addSystemRecommendation("[INFO] System needs attention, schedule maintenance for
optimal performance");
    } else {
        report.addSystemRecommendation("[ACTION] System requires comprehensive maintenance and
potential upgrades");
    }

    // Additional recommendations based on device mix
    long totalDevices = report.getTotalDevices();
    if (totalDevices > 10) {
        report.addSystemRecommendation("[TIP] Consider smart power strips for large device
setups");
    }

    report.addSystemRecommendation("[SCHEDULE] Monthly health checks for proactive
maintenance");
    report.addSystemRecommendation("[SETUP] Enable device notifications for real-time health
monitoring");
}
```

Intelligent Recommendation System:
- Priority alerts: Critical and warning device counts
- System-level advice: Based on overall health score
- Scalability tips: Special advice for large device setups (>10 devices)
- Proactive maintenance: Monthly check reminders
- Feature suggestions: Enable notifications for better monitoring

The DeviceHealthService provides predictive maintenance, comprehensive diagnostics, and professional-grade health monitoring for smart home systems!

EnergyManagementService.java - Energy Tracking and Cost Analysis

File Location: `src/main/java/com/smarthome/service/EnergyManagementService.java`

Overview

The EnergyManagementService class is like a smart electricity meter and financial advisor combined into one. It tracks every unit of electricity consumed by your smart home devices, calculates accurate electricity bills using real-world slab-based pricing (like Indian electricity boards), provides detailed cost breakdowns, and offers personalized energy-saving tips. Think of it as your personal energy accountant that helps you understand, manage, and optimize your electricity consumption and costs.

Package Declaration and Imports

java

```
package com.smarthome.service;

import com.smarthome.model.Customer;
import com.smarthome.model.Gadget;
import com.smarthome.model.DeletedDeviceEnergyRecord;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
```

Explanation:
- Service package: Business logic layer for energy management
- Model imports: Customer, Gadget, and DeletedDeviceEnergyRecord for comprehensive energy tracking
- Time handling: LocalDateTime for timestamps, DateTimeFormatter for user-friendly date displays
- Collections: List for managing device data and energy records
- No database imports: This is a calculation and reporting service, not a data storage service

Energy Report Data Structure
EnergyReport Inner Class
java

```
public static class EnergyReport {
    private double totalEnergyKWh;
    private double totalCostRupees;
    private String reportPeriod;
    private List<Gadget> devices;

    public EnergyReport(double totalEnergyKWh, double totalCostRupees, String reportPeriod,
List<Gadget> devices) {
        this.totalEnergyKWh = totalEnergyKWh;
        this.totalCostRupees = totalCostRupees;
        this.reportPeriod = reportPeriod;
        this.devices = devices;
    }
}
```

Energy Report Components:

Field	Type	Purpose
totalEnergyKWh	double	Total electricity consumed in kilowatt-hours
totalCostRupees	double	Total electricity bill in Indian Rupees
reportPeriod	String	Time period for the report (e.g., "January 2024")
devices	List<Gadget>	All devices included in the energy calculation

Why This Structure:
- Comprehensive data: Includes both energy consumption and financial cost
- Time awareness: Knows which period the report covers
- Device tracking: Maintains reference to all devices for detailed analysis
- Currency specific: Uses Rupees reflecting the Indian market focus

Comprehensive Energy Report Generation
Complete Energy Calculation
java

```
public EnergyReport generateEnergyReport(Customer customer) {
    List<Gadget> devices = customer.getGadgets();
    double totalEnergyKWh = 0.0;

    for (Gadget device : devices) {
        double currentSessionEnergy = 0.0;
        if (device.isOn() && device.getLastOnTime() != null) {
            double currentSessionHours = device.getCurrentSessionUsageHours();
            currentSessionEnergy = (device.getPowerRatingWatts() / 1000.0)  currentSessionHours;
        }

        totalEnergyKWh += device.getTotalEnergyConsumedKWh() + currentSessionEnergy;
    }

    double deletedDeviceEnergy = customer.getTotalDeletedDeviceEnergyForCurrentMonth();
    totalEnergyKWh += deletedDeviceEnergy;

    double totalCost = calculateSlabBasedCost(totalEnergyKWh);
    String reportPeriod = "Monthly Report - " +
LocalDateTime.now().format(DateTimeFormatter.ofPattern("MMMM yyyy"));
```

```
    return new EnergyReport(totalEnergyKWh, totalCost, reportPeriod, devices);
}
```

Smart Energy Calculation Process:

1. Current Active Devices

- Historical energy: `device.getTotalEnergyConsumedKWh()` from previous usage
- Current session: Real-time calculation for devices currently running
- Formula: `(Power in Watts ÷ 1000) × Hours = kWh`

2. Deleted Device Integration

- Historical preservation: Includes energy from deleted devices in current month
- Complete tracking: No energy consumption is lost from billing
- Month-specific: Only includes current month's deleted device energy

3. Cost Calculation

- Slab-based pricing: Uses realistic electricity board pricing structure
- Accurate billing: Reflects real-world electricity costs

4. Report Formatting

- User-friendly dates: "January 2024" instead of technical formats
- Complete package: Returns structured data with all necessary information

Advanced Slab-Based Cost Calculation
Indian Electricity Board Pricing Structure
java

```
public double calculateSlabBasedCost(double totalKWh) {
    double totalCost = 0.0;

    if (totalKWh <= 30) {
        totalCost = totalKWh  1.90;
    } else if (totalKWh <= 75) {
        totalCost = 30  1.90 + (totalKWh - 30)  3.0;
    } else if (totalKWh <= 125) {
        totalCost = 30  1.90 + 45  3.0 + (totalKWh - 75)  4.50;
    } else if (totalKWh <= 225) {
        totalCost = 30  1.90 + 45  3.0 + 50  4.50 + (totalKWh - 125)  6.0;
    } else if (totalKWh <= 400) {
        totalCost = 30  1.90 + 45  3.0 + 50  4.50 + 100  6.0 + (totalKWh - 225)  8.75;
    } else {
        totalCost = 30  1.90 + 45  3.0 + 50  4.50 + 100  6.0 + 175  8.75 + (totalKWh - 400)
9.75;
    }

    return Math.round(totalCost  100.0) / 100.0;
}
```

Electricity Pricing Slabs (Indian Rupees per kWh):

Slab Range	Units (kWh)	Rate per Unit (₹)	Cumulative Cost Calculation
Slab 1	0-30	₹1.90	First 30 units × ₹1.90
Slab 2	31-75	₹3.00	Next 45 units × ₹3.00
Slab 3	76-125	₹4.50	Next 50 units × ₹4.50
Slab 4	126-225	₹6.00	Next 100 units × ₹6.00
Slab 5	226-400	₹8.75	Next 175 units × ₹8.75
Slab 6	400+	₹9.75	Remaining units × ₹9.75

Mathematical Logic:

- Cumulative calculation: Each slab applies only to units within that range

- Progressive pricing: Higher consumption = higher per-unit rates
- Real-world accuracy: Based on actual Indian electricity board tariffs
- Precision rounding: `Math.round(totalCost  100.0) / 100.0` ensures 2 decimal places

Example Calculation for 150 kWh:
Slab 1: 30 units × ₹1.90 = ₹57.00
Slab 2: 45 units × ₹3.00 = ₹135.00
Slab 3: 50 units × ₹4.50 = ₹225.00
Slab 4: 25 units × ₹6.00 = ₹150.00
Total: ₹567.00

Professional Cost Breakdown Display
Detailed Slab-wise Breakdown
java

```
public String getSlabBreakdown(double totalKWh) {
    StringBuilder breakdown = new StringBuilder();
    breakdown.append("\n=== Energy Cost Breakdown (Slab-wise) ===\n");
    breakdown.append("+---------------+----------+----------+----------+\n");
    breakdown.append(String.format("| %-13s | %-8s | %-8s | %-8s |\n", "Slab Range", "Units",
"Rate/Unit", "Cost"));
    breakdown.append("+---------------+----------+----------+----------+\n");

    double remainingKWh = totalKWh;
    double totalCost = 0.0;

    if (remainingKWh > 0) {
        double slab1 = Math.min(remainingKWh, 30);
        double cost1 = slab1  1.90;
        totalCost += cost1;
        breakdown.append(String.format("| %-13s | %8.2f | %8.2f | %8.2f |\n", "0-30 kWh", slab1,
1.90, cost1));
        remainingKWh -= slab1;
    }
    // ... continues for all slabs

    breakdown.append("+---------------+----------+----------+----------+\n");
    breakdown.append(String.format("| %-13s | %8.2f | %-8s | %8.2f |\n", "TOTAL", totalKWh, "-",
totalCost));
    breakdown.append("+---------------+----------+----------+----------+\n");

    return breakdown.toString();
}
```

Professional Table Format:
=== Energy Cost Breakdown (Slab-wise) ===

Slab Range	Units	Rate/Unit	Cost
0-30 kWh	30.00	1.90	57.00
31-75 kWh	45.00	3.00	135.00
76-125 kWh	50.00	4.50	225.00
126-225 kWh	25.00	6.00	150.00
TOTAL	150.00	-	**567.00**

StringBuilder Benefits:
- Memory efficient: Better than string concatenation with +
- Professional formatting: Aligned columns and borders
- Complete breakdown: Shows exactly how the bill was calculated
- Transparency: Users understand where every rupee comes from

Comprehensive Device Energy Display
Detailed Device Energy Analysis

java

```
public void displayDeviceEnergyUsage(Customer customer) {
    List<Gadget> devices = customer.getGadgets();

    System.out.println("\n=== Device Energy Usage Details ===");
    System.out.println("+-------------------------+---------+---------+------------+-----------
--+-------------+");
    System.out.printf("| %-23s | %-7s | %-7s | %-11s | %-11s | %-11s |\n",
                      "Device", "Power", "Status", "Usage Time", "Energy(kWh)", "Cost(Rs.)");
    System.out.println("+-------------------------+---------+---------+------------+-----------
--+-------------+");

    // Display active devices
    for (Gadget device : devices) {
        double totalEnergy = device.getTotalEnergyConsumedKWh();

        if (device.isOn() && device.getLastOnTime() != null) {
            double currentSessionHours = device.getCurrentSessionUsageHours();
            double currentSessionEnergy = (device.getPowerRatingWatts() / 1000.0)
currentSessionHours;
            totalEnergy += currentSessionEnergy;
        }

        double deviceCost = calculateSlabBasedCost(totalEnergy);
        String status = device.isOn() ? "RUNNING" : "OFF";
        String deviceName = String.format("%s %s (%s)", device.getType(), device.getModel(),
device.getRoomName());
        if (deviceName.length() > 23) {
            deviceName = deviceName.substring(0, 20) + "...";
        }

        System.out.printf("| %-23s | %7.0fW | %-7s | %11s | %11.3f | %11.2f |\n",
                          deviceName,
                          device.getPowerRatingWatts(),
                          status,
                          device.getCurrentUsageTimeFormatted(),
                          totalEnergy,
                          deviceCost);

        if (device.isOn() && device.getLastOnTime() != null) {
            System.out.printf("| %-23s | %-7s | %-7s | %-11s | %-11s | %-11s |\n",
                              "  Current Session:", "", "", String.format("%.2fh",
device.getCurrentSessionUsageHours()), "", "");
        }
    }
}
```

Professional Device Report Format:

=== Device Energy Usage Details ===

Device	Power	Status	Usage Time	Energy(kWh)	Cost(Rs.)
TV Samsung 55" (Living)	150W	RUNNING	25h 30m	3.825	11.48
AC Voltas 1.5T (Bedroom	1500W	OFF	10h 15m	15.375	46.13
Light Philips (Kitchen)	60W	OFF	5h 45m	0.345	0.66

Smart Display Features:

- Real-time status: Shows RUNNING vs OFF for each device
- Current session tracking: Additional line for devices currently running
- Cost per device: Individual device cost calculation
- Truncated names: Long device names abbreviated to fit table
- Precise energy: 3 decimal places for accurate tracking

Deleted Device Energy Integration

java

```
// Display deleted devices energy consumption for current month
double deletedDeviceEnergy = customer.getTotalDeletedDeviceEnergyForCurrentMonth();
```

```
if (deletedDeviceEnergy > 0) {
    System.out.println("+------------------------+---------+---------+------------+----------
--+-------------+");
    double deletedDeviceCost = calculateSlabBasedCost(deletedDeviceEnergy);
    System.out.printf("| %-23s | %-7s | %-7s | %-11s | %11.3f | %11.2f |\n",
                      "[DELETED DEVICES]", "-", "DELETED", "-", deletedDeviceEnergy,
deletedDeviceCost);
    System.out.printf("| %-23s | %-7s | %-7s | %-11s | %-11s | %-11s |\n",
                      "  (Historical data)", "", "", "", "", "");
}
```

Complete Energy Tracking:
- No lost data: Deleted devices still counted in current month billing
- Clear identification: [DELETED DEVICES] clearly marked
- Historical context: Shows this is preserved historical data
- Accurate billing: Ensures users pay for all energy consumed

Detailed Deleted Device Breakdown

java

```
public void displayDeletedDeviceBreakdown(Customer customer) {
    LocalDateTime now = LocalDateTime.now();
    String currentMonth = now.getYear() + "-" + String.format("%02d", now.getMonthValue());

    List<DeletedDeviceEnergyRecord> deletedDevices = customer.getDeletedDeviceEnergyRecords();
    List<DeletedDeviceEnergyRecord> currentMonthDeleted = deletedDevices.stream()
            .filter(record -> currentMonth.equals(record.getDeletionMonth()))
            .toList();

    if (!currentMonthDeleted.isEmpty()) {
        System.out.println("\n=== Deleted Devices Energy Breakdown (This Month) ===");
        System.out.println("+------------------------+---------+-------------+-------------+---
-------------------+");
        System.out.printf("| %-23s | %-7s | %-11s | %-11s | %-20s |\n",
                          "Deleted Device", "Power", "Usage Time", "Energy(kWh)", "Deletion
Date");
        System.out.println("+------------------------+---------+-------------+-------------+---
-------------------+");

        for (DeletedDeviceEnergyRecord record : currentMonthDeleted) {
            String deviceName = String.format("%s %s (%s)", record.getDeviceType(),
record.getDeviceModel(), record.getRoomName());
            if (deviceName.length() > 23) {
                deviceName = deviceName.substring(0, 20) + "...";
            }

            String deletionDate =
record.getDeletionTime().format(DateTimeFormatter.ofPattern("dd-MM HH:mm"));

            System.out.printf("| %-23s | %7.0fW | %-11s | %11.3f | %-20s |\n",
                              deviceName,
                              record.getPowerRatingWatts(),
                              record.getFormattedUsageTime(),
                              record.getTotalEnergyConsumedKWh(),
                              deletionDate);
        }
        System.out.println("+------------------------+---------+-------------+-------------+---
-------------------+");
    }
}
```

Smart Data Filtering:
- Month-specific: Only shows deleted devices from current month
- Stream processing: Uses modern Java streams for filtering
- Complete details: Shows power, usage time, energy, and deletion timestamp
- User-friendly dates: "15-01 14:30" format instead of technical timestamps

Intelligent Energy Efficiency System
Personalized Energy-Saving Tips
java

```
public String getEnergyEfficiencyTips(double totalKWh) {
    StringBuilder tips = new StringBuilder();
    tips.append("\n=== Energy Efficiency Tips ===\n");

    if (totalKWh > 300) {
        tips.append("[HIGH USAGE ALERT] Your consumption is quite high!\n");
        tips.append("- Consider using Timer functions to auto-schedule device operations\n");
        tips.append("- Turn off devices when not in use\n");
        tips.append("- Use energy-efficient appliances\n");
    } else if (totalKWh > 150) {
        tips.append("[MODERATE USAGE] You're doing well, but there's room for improvement!\n");
        tips.append("- Use Timer scheduling for AC and Geyser during off-peak hours\n");
        tips.append("- Consider LED lighting for better efficiency\n");
    } else {
        tips.append("[EXCELLENT] Great job on managing your energy consumption!\n");
        tips.append("- Keep up the good work with energy-conscious usage\n");
    }

    tips.append("- Peak hours (6-10 PM): Avoid using high-power devices\n");
    tips.append("- Use our Calendar Events to schedule energy-intensive operations\n");

    return tips.toString();
}
```

Smart Recommendation Categories:

Usage Level	kWh Range	Alert Level	Recommendations
High Usage	300+ kWh	HIGH ALERT	Timer functions, turn off unused devices, energy-efficient appliances
Moderate Usage	150-300 kWh	IMPROVEMENT	Timer scheduling, LED lighting, off-peak usage
Excellent Usage	<150 kWh	EXCELLENT	Keep up good work, maintain energy-conscious habits

Universal Tips:
- Peak hour awareness: Avoid 6-10 PM high-power device usage
- Smart integration: Leverage Calendar Events for energy optimization
- Actionable advice: Specific, implementable recommendations

Real-World Usage Examples
Example 1: Monthly Energy Report Generation

java

```
EnergyManagementService energyService = new EnergyManagementService();

// Generate comprehensive energy report for customer
Customer customer = getLoggedInCustomer();
EnergyReport report = energyService.generateEnergyReport(customer);

// Display results
System.out.println("=== " + report.getReportPeriod() + " ===");
System.out.printf("Total Energy Consumed: %.2f kWh\n", report.getTotalEnergyKWh());
System.out.printf("Total Electricity Cost: ₹%.2f\n", report.getTotalCostRupees());

// Show detailed cost breakdown
String breakdown = energyService.getSlabBreakdown(report.getTotalEnergyKWh());
System.out.println(breakdown);

// Show energy efficiency tips
String tips = energyService.getEnergyEfficiencyTips(report.getTotalEnergyKWh());
System.out.println(tips);
```

Complete Energy Analysis:
- Total consumption: Includes active + deleted device energy
- Accurate cost: Real slab-based pricing calculation
- Detailed breakdown: Shows exactly how bill was calculated
- Personalized tips: Energy-saving advice based on usage level

Example 2: Device-Level Energy Analysis

java

```
// Show detailed device energy usage
energyService.displayDeviceEnergyUsage(customer);

// Results in professional table format:
// - Individual device power consumption
// - Current running status
// - Total usage time and energy
// - Individual device cost
// - Current session information for running devices
// - Historical data from deleted devices
```

Example 3: Cost Calculation for Different Usage Levels

java

```
// Calculate costs for different consumption levels
double lowUsage = 45.0;    // kWh
double moderateUsage = 180.0;
double highUsage = 350.0;

double lowCost = energyService.calculateSlabBasedCost(lowUsage);
double moderateCost = energyService.calculateSlabBasedCost(moderateUsage);
double highCost = energyService.calculateSlabBasedCost(highUsage);

System.out.printf("Low usage (45 kWh): ₹%.2f\n", lowCost);        // ₹135.00
System.out.printf("Moderate usage (180 kWh): ₹%.2f\n", moderateCost); // ₹747.00
System.out.printf("High usage (350 kWh): ₹%.2f\n", highCost);      // ₹2,558.75
```

Realistic Cost Comparison:
- Low usage: Basic consumption with lower per-unit rates
- Moderate usage: Average household consumption
- High usage: Heavy consumption with higher slab rates

Advanced Features Summary

🔢 Mathematical Excellence
- Precise calculations: Real-world slab-based pricing implementation
- Progressive pricing: Accurate utility billing simulation
- Rounding precision: Financial-grade accuracy with proper rounding
- Complex formulas: Multi-slab cumulative cost calculations

📊 Professional Reporting
- Structured data: EnergyReport class for organized information
- Table formatting: Professional ASCII tables with aligned columns
- Complete tracking: Includes active, running, and deleted device energy
- Time-aware analysis: Monthly, current session, and historical data

💡 Smart Analytics
- Usage-based recommendations: Personalized energy-saving tips
- Real-time calculations: Includes currently running device sessions
- Historical integration: Preserves deleted device energy for accurate billing
- Peak hour awareness: Load management and cost optimization advice

🏠 Real-World Accuracy
- Indian pricing model: Based on actual electricity board tariffs
- Currency-specific: Uses Indian Rupees for cost calculations
- Practical advice: Actionable energy efficiency recommendations
- Complete billing: No energy consumption is lost or unaccounted

Summary for Beginners

This `EnergyManagementService` class demonstrates advanced mathematical modeling and real-world business logic:

🎯 Why This Class Exists:
- Accurate billing: Calculate exact electricity costs using real utility pricing
- Energy awareness: Help users understand their consumption patterns
- Cost optimization: Provide actionable advice for reducing electricity bills
- Complete tracking: Account for all energy consumption including deleted devices

📐 Advanced Programming Concepts:
- Complex calculations: Multi-slab progressive pricing algorithms
- Professional formatting: StringBuilder for efficient text processing
- Data integration: Combines multiple data sources for comprehensive reporting
- Stream processing: Modern Java techniques for data filtering and analysis

💡 Real-World Applications:
- Utility billing systems: Electricity, gas, and water utility companies
- Energy management platforms: Commercial and residential energy monitoring
- Smart building systems: Enterprise energy optimization
- IoT device management: Industrial and consumer device energy tracking

📚 Programming Concepts You Learned:
- Mathematical modeling: Complex progressive pricing calculations
- StringBuilder usage: Efficient string building for large text
- Professional formatting: Aligned tables and structured reports
- Data aggregation: Combining multiple sources for comprehensive analysis
- Conditional pricing: Different rates based on consumption levels
- Precision handling: Financial-grade rounding and accuracy
- Stream filtering: Modern Java collection processing
- Time-based calculations: Month-specific data analysis

The EnergyManagementService shows how modern applications handle complex business rules, financial calculations, and professional reporting to solve real-world problems!

GadgetService.java - Device Validation and Factory

File Location: `src/main/java/com/smarthome/service/GadgetService.java`

Overview

The GadgetService class is like a comprehensive database of all valid smart home devices and rooms in the Indian market. It acts as a smart validator and factory that knows exactly which device models exist, which rooms are valid in Indian homes, and ensures that only legitimate devices can be added to the smart home system. Think of it as a knowledgeable electronics store manager who knows every brand, every model, and every proper room configuration - preventing users from entering invalid or non-existent device information.

Package Declaration and Imports

```
java
package com.smarthome.service;

import com.smarthome.model.Gadget;
import java.util.Arrays;
import java.util.List;
```

Explanation:
- Service package: Business logic layer for device validation and creation
- Gadget model: Imports the Gadget class for device creation
- Collections: Arrays and List for managing validation data
- Minimalist imports: Only essential imports needed for validation logic

Comprehensive Room Validation System
Indian Home Room Configuration
java

```
private static final List<String> VALID_ROOMS = Arrays.asList(
    "Living Room", "Hall", "Drawing Room", "Family Room", "Sitting Room",
    "Master Bedroom", "Bedroom 1", "Bedroom 2", "Bedroom 3", "Guest Room",
    "Kids Room", "Parents Room", "Childrens Room",
    "Kitchen", "Dining Room", "Breakfast Area", "Pantry", "Store Room",
    "Master Bathroom", "Common Bathroom", "Guest Bathroom", "Powder Room",
    "Balcony", "Terrace", "Garden", "Courtyard", "Entrance", "Porch",
    "Garage", "Parking", "Utility Room", "Laundry Room",
    "Study Room", "Office Room", "Library", "Home Office",
    "Home Theater", "Entertainment Room", "Prayer Room", "Pooja Room",
    "Servant Room", "Driver Room", "Security Room", "Storage Room",
    "Verandah", "Chowk", "Angan", "Baithak", "Otla"
);
```

Room Categories and Indian Context:

Category	Rooms	Cultural Context
Living Areas	Living Room, Hall, Drawing Room, Family Room, Sitting Room	Central gathering spaces in Indian homes
Sleeping Areas	Master Bedroom, Bedroom 1-3, Guest Room, Kids Room, Parents Room	Multiple generations living together
Kitchen Areas	Kitchen, Dining Room, Breakfast Area, Pantry, Store Room	Extended family cooking and dining needs
Bathrooms	Master, Common, Guest Bathroom, Powder Room	Multiple bathrooms for large families
Outdoor Areas	Balcony, Terrace, Garden, Courtyard, Entrance, Porch	Important outdoor living spaces in Indian homes
Utility Areas	Garage, Parking, Utility Room, Laundry Room	Practical spaces for household management
Work Areas	Study Room, Office Room, Library, Home Office	Education and work-from-home spaces
Entertainment	Home Theater, Entertainment Room	Modern entertainment needs
Religious Areas	Prayer Room, Pooja Room	Essential spiritual spaces in Indian homes
Service Areas	Servant Room, Driver Room, Security Room, Storage Room	Traditional household staff accommodation
Traditional Indian	Verandah, Chowk, Angan, Baithak, Otla	Cultural architectural elements

Cultural Intelligence:
- Indian terminology: Includes traditional Indian room names like "Chowk", "Angan", "Baithak"
- Extended family living: Multiple bedrooms for joint family systems
- Religious consideration: Dedicated prayer/pooja rooms
- Service accommodation: Rooms for household staff (common in Indian homes)
- Outdoor living: Emphasis on terraces, balconies, courtyards

Extensive Device Model Validation

Television Models (30 Brands)

java

```
private static final List<String> VALID_TV_MODELS = Arrays.asList(
    "MI", "Realme", "OnePlus", "TCL", "Samsung", "LG", "Sony", "Panasonic",
    "Haier", "Thomson", "Kodak", "Motorola", "Nokia", "Videocon", "BPL",
    "Micromax", "Intex", "Vu", "Shinco", "Daiwa", "Akai", "Sanyo", "Lloyd",
    "Hitachi", "Toshiba", "Sharp", "Philips", "Iffalcon", "Coocaa", "MarQ"
);
```

Brand Categories:
- Premium International: Samsung, LG, Sony, Panasonic (₹40,000+)
- Chinese Value: MI, Realme, OnePlus, TCL (₹15,000-₹35,000)
- Indian Budget: Micromax, Intex, Videocon, BPL (₹8,000-₹20,000)
- Traditional Electronics: Hitachi, Toshiba, Sharp, Philips (₹25,000-₹50,000)

Air Conditioner Models (33 Brands)

java

```
private static final List<String> VALID_AC_MODELS = Arrays.asList(
    "Voltas", "Blue Star", "Godrej", "Lloyd", "Carrier", "Daikin", "Hitachi",
    "LG", "Samsung", "Panasonic", "Whirlpool", "O General", "Mitsubishi",
    "Haier", "IFB", "Sansui", "Koryo", "MarQ", "Onida", "Kelvinator",
    "Bajaj", "Usha", "Orient", "Maharaja Whiteline", "Hindware", "Havells",
    "Videocon", "BPL", "Electrolux", "Bosch", "Siemens", "Midea", "Gree"
);
```

AC Market Segments:
- Premium Inverter: Daikin, Mitsubishi, Carrier (₹45,000+)
- Popular Choice: Voltas, Blue Star, LG, Samsung (₹25,000-₹45,000)
- Budget Options: Lloyd, Koryo, MarQ (₹18,000-₹30,000)
- Indian Brands: Voltas, Blue Star, Godrej (Strong Indian presence)

Fan Models (27 Brands)

java

```
private static final List<String> VALID_FAN_MODELS = Arrays.asList(
    "Havells", "Bajaj", "Usha", "Orient", "Crompton", "Luminous", "Atomberg",
    "Polycab", "Anchor", "Syska", "V-Guard", "Khaitan", "Surya", "Almonard",
    "Maharaja Whiteline", "Hindware", "Rico", "GM", "Standard", "Activa",
    "Singer", "Orpat", "Seema", "Citron", "Lazer", "Finolex", "Replay"
);
```

Fan Categories:
- Premium Ceiling: Havells, Bajaj, Usha, Orient (₹2,000-₹8,000)
- Smart/BLDC: Atomberg, Crompton (₹3,000-₹12,000)
- Industrial: Almonard (₹15,000+)
- Budget Options: Rico, GM, Standard (₹800-₹2,500)

Smart Lighting Models (18 Brands)

java

```
private static final List<String> VALID_LIGHT_MODELS = Arrays.asList(
    "Philips Hue", "MI", "Syska", "Havells", "Wipro", "Bajaj", "Orient",
    "Crompton", "Polycab", "V-Guard", "Anchor", "Luminous", "Eveready",
    "Osram", "Godrej", "Schneider Electric", "Legrand", "Panasonic"
);
```

Lighting Categories:
- Smart/Connected: Philips Hue, MI (₹500-₹5,000)
- LED Premium: Syska, Havells, Wipro (₹200-₹1,500)
- Electrical Brands: Polycab, V-Guard, Anchor (₹100-₹800)
- Traditional: Eveready, Osram (₹80-₹500)

Smart Validation Methods
Room Validation with Case-Insensitive Matching
java

```
public boolean isValidRoom(String roomName) {
    return roomName != null && VALID_ROOMS.stream()
            .anyMatch(room -> room.equalsIgnoreCase(roomName));
}
```

Validation Features:
- Null safety: Checks for null input to prevent errors
- Stream processing: Uses modern Java streams for efficient searching
- Case-insensitive: Accepts "living room", "LIVING ROOM", "Living Room"
- Exact matching: Prevents invalid room names from entering the system

Device-Specific Model Validation
java

```
public boolean isValidTVModel(String model) {
    return model != null && VALID_TV_MODELS.stream()
            .anyMatch(validModel -> validModel.equalsIgnoreCase(model));
}

public boolean isValidACModel(String model) {
    return model != null && VALID_AC_MODELS.stream()
            .anyMatch(validModel -> validModel.equalsIgnoreCase(model));
}

// Similar methods for all device types...
```

Individual Validation Methods:
- Each device type has its own validation method
- Consistent pattern across all device types
- Null safety and case-insensitive matching
- Easy to maintain and extend

Universal Model Validation with Type-Aware Logic

```
java
public boolean isValidModel(String type, String model) {
    if (type == null || model == null) {
        return false;
    }
    switch (type.toUpperCase()) {
        case "TV":
            return isValidTVModel(model);
        case "AC":
        case "AIR_CONDITIONER":
            return isValidACModel(model);
        case "FAN":
            return isValidFanModel(model);
        case "LIGHT":
        case "SMART_LIGHT":
            return isValidLightModel(model);
        case "SWITCH":
        case "SMART_SWITCH":
            return isValidSwitchModel(model);
        // ... continues for all device types
        default:
            return false;
    }
}
```

Smart Type Recognition:

- Multiple aliases: Recognizes both "AC" and "AIR_CONDITIONER"
- Case normalization: Converts input to uppercase for consistent comparison
- Comprehensive coverage: Handles all supported device types
- Fail-safe default: Returns false for unknown device types

Supported Device Type Aliases:

Primary Type	Aliases	Examples
TV	TV	Samsung, LG, MI, Sony
AC	AC, AIR_CONDITIONER	Voltas, Daikin, Blue Star
LIGHT	LIGHT, SMART_LIGHT	Philips Hue, MI, Syska
SWITCH	SWITCH, SMART_SWITCH	Anchor, Havells, Legrand
CAMERA	CAMERA, SECURITY_CAMERA	MI, CP Plus, Hikvision
DOOR_LOCK	DOOR_LOCK, SMART_LOCK	Godrej, Yale, Samsung
VACUUM	VACUUM, ROBOTIC_VACUUM, ROBO_VAC_MOP	MI Robot, Eureka Forbes
REFRIGERATOR	REFRIGERATOR, FRIDGE	LG, Samsung, Whirlpool

Factory Pattern Implementation
Secure Gadget Creation
Java

```
public Gadget createGadget(String type, String model, String roomName) {
    if (!isValidRoom(roomName)) {
        throw new IllegalArgumentException("Invalid room name. Valid rooms: " + VALID_ROOMS);
    }

    if (!isValidModel(type, model)) {
        throw new IllegalArgumentException("Invalid model for " + type);
    }

    return new Gadget(type.toUpperCase(), model, roomName);
}
```

Factory Pattern Benefits:
- Centralized creation: All gadget creation goes through this method
- Validation guarantee: Can't create invalid gadgets
- Type normalization: Ensures consistent device type formatting
- Error feedback: Clear error messages for invalid inputs

Security Features:
- Input validation: Prevents creation of non-existent devices
- Exception handling: Throws meaningful exceptions for invalid inputs
- Data integrity: Ensures only valid data enters the system
- Consistent formatting: Type converted to uppercase for standardization

Information Retrieval System
Getting Valid Options for Users

java

```
public List<String> getValidRooms() {
    return VALID_ROOMS;
}

public List<String> getValidModelsForType(String type) {
    if (type == null) {
        return Arrays.asList();
    }

    switch (type.toUpperCase()) {
        case "TV":
            return VALID_TV_MODELS;
        case "AC":
```

```
        case "AIR_CONDITIONER":
            return VALID_AC_MODELS;
        // ... continues for all types
        default:
            return Arrays.asList();
    }
}
```

User-Friendly Information Access:
- Complete room list: Returns all 46 valid room names
- Type-specific models: Returns only relevant models for chosen device type
- Empty list fallback: Returns empty list for unknown types (no null pointer exceptions)
- UI integration ready: Perfect for populating dropdown menus and selection lists

Formatted Display Methods

java

```
public String getFormattedValidRooms() {
    return String.join(", ", VALID_ROOMS);
}

public String getFormattedValidModels(String type) {
    List<String> models = getValidModelsForType(type);
    return models.isEmpty() ? "No valid models" : String.join(", ", models);
}
```

Professional Formatting:
- Comma-separated lists: Ready for display in user interfaces
- Graceful handling: Shows "No valid models" for invalid types
- String.join() usage: Modern Java approach to string concatenation

Example Output:
Valid Rooms: Living Room, Hall, Drawing Room, Family Room, Master Bedroom...
Valid TV Models: MI, Realme, OnePlus, TCL, Samsung, LG, Sony, Panasonic...
Valid AC Models: Voltas, Blue Star, Godrej, Lloyd, Carrier, Daikin...

Real-World Usage Examples

Example 1: Creating a Valid Device

java

```
GadgetService gadgetService = new GadgetService();

// Attempt to create a valid TV
try {
    Gadget tv = gadgetService.createGadget("TV", "Samsung", "Living Room");
    System.out.println("TV created successfully: " + tv.toString());
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}

// Attempt to create an invalid device
try {
    Gadget invalidTV = gadgetService.createGadget("TV", "NonExistentBrand", "Living Room");
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage()); // "Invalid model for TV"
}
```

Example 2: Validating User Input

java

```
// Validate room before allowing user to proceed
String userRoom = "master bedroom"; // User input
```

```
if (gadgetService.isValidRoom(userRoom)) {
    System.out.println("Valid room selected");
} else {
    System.out.println("Invalid room. Valid options: " +
gadgetService.getFormattedValidRooms());
}

// Validate device model
String deviceType = "AC";
String userModel = "voltas"; // User input
if (gadgetService.isValidModel(deviceType, userModel)) {
    System.out.println("Valid AC model selected");
} else {
    System.out.println("Invalid AC model. Valid options: " +
gadgetService.getFormattedValidModels(deviceType));
}
```

Example 3: Populating UI Dropdowns

java

```
// Get all valid rooms for dropdown menu
List<String> roomOptions = gadgetService.getValidRooms();
// UI code would populate dropdown with these options

// Get valid models for selected device type
String selectedDeviceType = "TV";
List<String> modelOptions = gadgetService.getValidModelsForType(selectedDeviceType);
// UI code would populate model dropdown with these options

// Format for display in help text
String helpText = "Valid rooms: " + gadgetService.getFormattedValidRooms();
```

Device Coverage and Market Analysis

Complete Device Category Coverage

Device Category	Brand Count	Market Coverage	Price Range
Television	30 brands	Budget to Premium	₹8,000 - ₹2,00,000
Air Conditioner	33 brands	Window to Inverter	₹18,000 - ₹80,000
Ceiling Fan	27 brands	Regular to BLDC	₹800 - ₹15,000
Smart Lighting	18 brands	LED to Smart Bulbs	₹80 - ₹5,000
Smart Switch	19 brands	Manual to WiFi	₹50 - ₹2,000
Security Camera	20 brands	Basic to AI-enabled	₹1,500 - ₹15,000
Smart Lock	20 brands	Keypad to Biometric	₹8,000 - ₹50,000
Water Heater	18 brands	Instant to Storage	₹3,000 - ₹25,000
Smart Doorbell	13 brands	Audio to Video	₹2,000 - ₹15,000
Robotic Vacuum	17 brands	Basic to Mapping	₹8,000 - ₹80,000
Air Purifier	19 brands	HEPA to Smart	₹5,000 - ₹60,000
Smart Speaker	13 brands	Basic to Premium	₹2,000 - ₹20,000

Indian Market Intelligence

Brand Distribution Analysis:
- International Brands: Samsung, LG, Sony, Panasonic, Philips (Premium segment)
- Chinese Brands: MI, Realme, OnePlus, TCL, Haier (Value segment)
- Indian Brands: Voltas, Blue Star, Bajaj, Havells, Godrej (Trusted local)
- Traditional Electronics: Videocon, BPL, Onida (Budget heritage brands)

Regional Preferences:
- North India: Voltas AC, Bajaj fans popular
- South India: Blue Star AC, Crompton fans preferred
- Urban Areas: Smart brands like MI, Realme gaining popularity
- Rural Areas: Traditional brands like Bajaj, Usha still dominant

Advanced Features Summary

🔍 Comprehensive Validation
- 46 room types: Complete coverage of Indian home architecture
- 300+ device models: Extensive brand recognition across all categories
- Case-insensitive matching: User-friendly input handling
- Null safety: Robust error prevention throughout

🏭 Factory Pattern Implementation
- Centralized creation: Single point for all device instantiation
- Validation guarantee: Impossible to create invalid devices
- Error feedback: Clear, actionable error messages
- Type normalization: Consistent data formatting

🎯 Market Intelligence
- Indian brand focus: Reflects actual Indian electronics market
- Price segment coverage: Budget to premium brands included
- Cultural awareness: Traditional Indian architectural terminology
- Regional relevance: Brands popular in Indian subcontinent

🔧 Developer-Friendly API
- Multiple access methods: Lists, formatted strings, individual validation
- UI integration ready: Perfect for dropdown population
- Exception handling: Meaningful error messages for debugging
- Extensible design: Easy to add new brands and room types

Summary for Beginners

This `GadgetService` class demonstrates professional data validation and factory design patterns:

🎯 Why This Class Exists:
- Data integrity: Prevents invalid device information from entering the system
- User experience: Provides helpful validation and suggestions
- Market relevance: Reflects real Indian electronics market
- Quality assurance: Ensures only legitimate devices are supported

📐 Design Patterns Used:
- Factory Pattern: Centralized, validated object creation
- Validation Pattern: Comprehensive input checking
- Constants Pattern: Immutable validation data
- Stream Processing: Modern Java collection handling

💡 Real-World Applications:
- E-commerce platforms: Product validation for electronics marketplaces
- IoT device management: Ensuring supported device compatibility
- Smart home systems: Validating device additions
- Inventory management: Verifying product catalog entries

📚 Programming Concepts You Learned:
- Static final collections: Immutable data structures with Arrays.asList()
- Stream API validation: Using anyMatch() for efficient searching
- Case-insensitive comparison: equalsIgnoreCase() for user-friendly matching

- Factory pattern: Centralized object creation with validation
- Exception handling: IllegalArgumentException for validation failures
- Switch statements: Efficient type-based routing
- String processing: String.join() for formatted output
- Defensive programming: Null checks and input validation

The GadgetService shows how modern applications implement data quality, market intelligence, and user-friendly validation to create reliable, professional software systems!

SmartHomeService.java - Central Orchestrator and Facade

File Location: `src/main/java/com/smarthome/service/SmartHomeService.java`

Overview

The SmartHomeService class is like the central command center of a smart home system. It acts as the main orchestrator that brings together all the specialized services (CustomerService, GadgetService, EnergyManagementService, etc.) and provides a unified, user-friendly interface for the entire smart home application. Think of it as the master controller that knows how to coordinate all the different subsystems - authentication, device management, energy tracking, health monitoring, and more. This class implements the Facade design pattern, hiding the complexity of multiple services behind a simple, cohesive API.

Package Declaration and Imports

java

```
package com.smarthome.service;

import com.smarthome.model.Customer;
import com.smarthome.model.Gadget;
import com.smarthome.model.DeletedDeviceEnergyRecord;
import com.smarthome.util.SessionManager;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;
```

Explanation:
- Service package: Central orchestrator in the service layer
- Model imports: All key model classes for comprehensive functionality
- SessionManager: User session state management
- Collections: ArrayList, HashMap, List, Map for data management
- Time handling: LocalDateTime, DateTimeFormatter, ChronoUnit for scheduling and formatting
- Comprehensive imports: Everything needed to coordinate all subsystems

Service Composition and Dependency Injection

Central Service Orchestration

java

```
public class SmartHomeService {

    private final CustomerService customerService;
    private final GadgetService gadgetService;
    private final SessionManager sessionManager;
    private final EnergyManagementService energyService;
    private final TimerService timerService;
    private final CalendarEventService calendarService;
    private final WeatherService weatherService;
    private final SmartScenesService smartScenesService;
    private final DeviceHealthService deviceHealthService;
```

```
    public SmartHomeService() {
        this.customerService = new CustomerService();
        this.gadgetService = new GadgetService();
        this.sessionManager = SessionManager.getInstance();
        this.energyService = new EnergyManagementService();
        this.timerService = TimerService.getInstance(customerService);
        this.calendarService = CalendarEventService.getInstance();
        this.weatherService = WeatherService.getInstance();
        this.smartScenesService = SmartScenesService.getInstance();
        this.deviceHealthService = DeviceHealthService.getInstance();
    }
```

Service Composition Strategy:

Service	**Pattern**	**Purpose**
CustomerService	Regular class	User authentication and management
GadgetService	Regular class	Device validation and creation
SessionManager	Singleton	User session state management
EnergyManagementService	Regular class	Energy tracking and billing
TimerService	Singleton with dependency	Device scheduling with database access
CalendarEventService	Singleton	Calendar event management
WeatherService	Singleton	Weather data integration
SmartScenesService	Singleton	Scene automation management
DeviceHealthService	Singleton	Device health monitoring

Design Pattern Benefits:
- Facade Pattern: Single point of access for complex subsystem
- Dependency Injection: All services injected through constructor
- Singleton Management: Properly handles singleton service instances
- Service Coordination: Orchestrates interactions between services

Authentication and User Management
Comprehensive User Registration
java

```
public boolean registerCustomer(String fullName, String email, String password, String
confirmPassword) {
    if (!password.equals(confirmPassword)) {
        System.out.println("Passwords do not match!");
        return false;
    }

    if (!customerService.isValidName(fullName)) {
        System.out.println("Invalid name! Name should contain only letters and spaces (minimum 2
characters).");
        return false;
    }

    if (!customerService.isValidEmail(email)) {
        System.out.println("Invalid email format!");
        return false;
    }

    if (!customerService.isValidPassword(password)) {
        System.out.println("[ERROR] Invalid password! Please ensure your password meets all
requirements:");
        System.out.println(customerService.getPasswordRequirements());
        return false;
    }
```

```
    boolean success = customerService.registerCustomer(fullName, email, password);
    if (success) {
        System.out.println("[SUCCESS] Thank you! Customer registration successful.");
    } else {
        System.out.println("[ERROR] Registration failed! Email might already be registered.");
    }

    return success;
}
```

Multi-Layer Validation Process:
1. Password confirmation: Ensures both passwords match
2. Name validation: Validates full name format and content
3. Email validation: Checks email format with regex patterns
4. Password strength: Validates against complex security requirements
5. Database registration: Attempts to create account in database
6. User feedback: Clear success/failure messages with guidance

Facade Pattern Benefits:
- Single method call: UI only needs one method for complete registration
- Comprehensive validation: All validation rules applied consistently
- User-friendly errors: Clear, actionable error messages
- Success confirmation: Positive feedback for successful registration

Secure Login with Session Management
java

```
public boolean loginCustomer(String email, String password) {
    Customer customer = customerService.authenticateCustomer(email, password);

    if (customer != null) {
        sessionManager.login(customer);
        return true;
    } else {
        return false;
    }
}
```

```
public void logout() {
   String currentUserEmail = sessionManager.getCurrentUser() != null ?
                    sessionManager.getCurrentUser().getEmail() : "unknown";
   sessionManager.logout();
   System.out.println("[SUCCESS] Logged out successfully from account: " + currentUserEmail);
   System.out.println("[INFO] You can now register a new account or login with different credentials.");
}
```

Session Management Integration:
- Authentication delegation: Uses CustomerService for password verification
- Session establishment: Automatically creates session on successful login
- Session cleanup: Properly clears session on logout
- User feedback: Informative logout messages with next steps

Device Management and Control
Intelligent Device Connection
java

```
public boolean connectToGadget(String type, String model, String roomName) {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("Please login first!");
        return false;
    }
```

```
    try {
        Customer currentUser = sessionManager.getCurrentUser();

        Gadget existingGadget = currentUser.findGadget(type, roomName);
        if (existingGadget != null) {
            System.out.println("A " + type + " already exists in " + roomName + ". You can only
have one " + type + " per room.");
            return false;
        }

        Gadget gadget = gadgetService.createGadget(type, model, roomName);
        gadget.ensurePowerRating();
        currentUser.addGadget(gadget);

        boolean updated = customerService.updateCustomer(currentUser);

        if (updated) {
            sessionManager.updateCurrentUser(currentUser);
            System.out.println("[SUCCESS] Successfully connected to " + gadget.getType() + " " +
gadget.getModel() + " in " + gadget.getRoomName());
            return true;
        } else {
            System.out.println("[ERROR] Failed to update customer data!");
            return false;
        }

    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
        return false;
    } catch (Exception e) {
        System.out.println("Unexpected error occurred. Please try again.");
        return false;
    }
}
```

Smart Device Addition Process:

1. Session validation: Ensures user is logged in
2. Duplicate prevention: Checks for existing device in same room
3. Factory creation: Uses GadgetService for validated device creation
4. Power rating setup: Ensures device has proper power consumption data
5. User association: Adds device to current user's device list
6. Database persistence: Saves changes to persistent storage
7. Session update: Updates current session with latest user data
8. Exception handling: Comprehensive error handling with user feedback

Advanced Device Viewing with Group Support

java

```
public List<Gadget> viewGadgets() {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("Please login first!");
        return null;
    }

    Customer currentUser = sessionManager.getCurrentUser();
    List<Gadget> allGadgets = new ArrayList<>();

    if (currentUser.isPartOfGroup()) {
        if (currentUser.getGadgets() != null) {
            allGadgets.addAll(currentUser.getGadgets());
        }

        int groupDeviceCount = 0;
        List<Customer> groupMemberObjects = new ArrayList<>();

        for (String memberEmail : currentUser.getGroupMembers()) {
            Customer groupMember = customerService.findCustomerByEmail(memberEmail);
            if (groupMember != null) {
```

```
                groupMemberObjects.add(groupMember);
            }
        }

        List<Gadget> accessibleGroupDevices =
currentUser.getAccessibleGroupDevices(groupMemberObjects);
        allGadgets.addAll(accessibleGroupDevices);
        groupDeviceCount = accessibleGroupDevices.size();

        System.out.println("\n=== Group Gadgets ===");
        System.out.println("[INFO] Group size: " + currentUser.getGroupSize() + " member(s) |
Admin: " + currentUser.getGroupCreator());
        System.out.println("[INFO] Your role: " + (currentUser.isGroupAdmin() ? "Admin" :
"Member"));
        System.out.println("[INFO] Showing your devices + devices you have permission to
access");
        if (groupDeviceCount > 0) {
            System.out.println("[INFO] You have access to " + groupDeviceCount + " group member
devices");
        } else {
            System.out.println("[INFO] No group devices shared with you. Ask admin for device
access permissions.");
        }
    } else {
        allGadgets = currentUser.getGadgets();
        System.out.println("\n=== Your Gadgets ===");
        System.out.println("[INFO] Showing only your personal devices (not part of any group)");
        System.out.println("[INFO] Use 'Group Management' to create a group and share devices
with others");
    }

    if (allGadgets == null || allGadgets.isEmpty()) {
        System.out.println("No gadgets found! Please connect to some gadgets first.");
        return allGadgets;
    }

    for (Gadget gadget : allGadgets) {
        gadget.ensurePowerRating();
    }

    displayAutoAlignedTable(allGadgets);

    return allGadgets;
}
```

Intelligent Device Aggregation:
- Personal devices: User's own devices always included
- Group devices: Devices from group members with permissions
- Permission checking: Only shows devices user has access to
- Group context: Clear information about group status and role
- Smart messaging: Different messages for group vs. individual users
- Power rating verification: Ensures all devices have proper energy data
- Professional display: Auto-aligned table format for optimal readability

Sophisticated Device Status Control
java

```
public boolean changeGadgetStatus(String gadgetType) {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("Please login first!");
        return false;
    }

    try {
        if (gadgetType == null || gadgetType.trim().isEmpty()) {
            System.out.println("Gadget type cannot be empty!");
            return false;
        }
```

```
        gadgetType = gadgetType.trim().toUpperCase();

        Customer currentUser = sessionManager.getCurrentUser();
        List<Gadget> gadgets = viewGadgets();

        if (gadgets == null || gadgets.isEmpty()) {
            System.out.println("No gadgets found! Please connect to some gadgets first.");
            return false;
        }

        Gadget targetGadget = null;
        Customer gadgetOwner = null;

        // Search in user's own devices first
        if (currentUser.getGadgets() != null) {
            for (Gadget gadget : currentUser.getGadgets()) {
                if (gadget.getType().equalsIgnoreCase(gadgetType)) {
                    targetGadget = gadget;
                    gadgetOwner = currentUser;
                    break;
                }
            }
        }

        // Search in group member devices if not found in own devices
        if (targetGadget == null && currentUser.isPartOfGroup()) {
            for (String memberEmail : currentUser.getGroupMembers()) {
                Customer member = customerService.findCustomerByEmail(memberEmail);
                if (member != null && member.getGadgets() != null) {
                    for (Gadget gadget : member.getGadgets()) {
                        if (gadget.getType().equalsIgnoreCase(gadgetType)) {
                            targetGadget = gadget;
                            gadgetOwner = member;
                            break;
                        }
                    }
                    if (targetGadget != null) break;
                }
            }
        }

        if (targetGadget == null) {
            System.out.println("Gadget type '" + gadgetType + "' not found!");
            System.out.println("Available gadgets: " + gadgets.stream()
                    .map(Gadget::getType)
                    .distinct()
                    .reduce((a, b) -> a + ", " + b)
                    .orElse("None"));
            return false;
        }

        targetGadget.ensurePowerRating();
        String previousStatus = targetGadget.getStatus();
        targetGadget.toggleStatus();
        String newStatus = targetGadget.getStatus();

        boolean updated = customerService.updateCustomer(gadgetOwner);

        if (gadgetOwner.getEmail().equals(currentUser.getEmail())) {
            sessionManager.updateCurrentUser(gadgetOwner);
        }

        if (updated) {
            if ("ON".equals(newStatus)) {
                System.out.println("[SUCCESS] Switched on successful");
            } else {
                System.out.println("[SUCCESS] Switched off successful");
            }

            System.out.println("\n=== All Gadgets Status ===");
```

```
            viewGadgets();
            return true;
        } else {
            targetGadget.setStatus(previousStatus);
            System.out.println("[ERROR] Failed to update gadget status!");
            return false;
        }

    } catch (Exception e) {
        System.out.println("Error changing gadget status. Please try again.");
        return false;
    }
}
```

Advanced Device Control Features:
- Multi-source search: Searches own devices first, then group devices
- Permission awareness: Respects device ownership and permissions
- Rollback capability: Reverts status change if database update fails
- Smart feedback: Shows available devices if target not found
- Session synchronization: Updates session if user's own device changed
- Complete refresh: Shows updated device list after status change

Professional Table Formatting System
Dynamic Table Layout Calculation
java

```
private void displayAutoAlignedTable(List<Gadget> allGadgets) {
    TableDimensions dimensions = calculateTableDimensions(allGadgets);
    TableFormatStrings formats = createTableFormatStrings(dimensions);

    System.out.println("Device List (Enter number to view detailed energy info):");
    System.out.println(formats.borderFormat);
    System.out.printf(formats.headerFormat + "\n", "", "Device", "Power", "Status", "Usage
Time", "Energy(kWh)");
    System.out.println(formats.borderFormat);

    displayTableRows(allGadgets, formats);
    System.out.println(formats.borderFormat);
}

private TableDimensions calculateTableDimensions(List<Gadget> allGadgets) {
    int numWidth = Math.max(2, String.valueOf(allGadgets.size()).length());
    int deviceWidth = "Device".length();
    int powerWidth = "Power".length();
    int statusWidth = "Status".length();
    int usageWidth = "Usage Time".length();
    int energyWidth = "Energy(kWh)".length();

    LocalDateTime now = LocalDateTime.now();
    final DateTimeFormatter timeFormatter = DateTimeFormatter.ofPattern("dd-MM HH:mm");

    for (Gadget gadget : allGadgets) {
        String deviceName = String.format("%s %s (%s)", gadget.getType(), gadget.getModel(),
gadget.getRoomName());
        deviceWidth = Math.max(deviceWidth, deviceName.length());

        String powerStr = String.format("%.0fW", gadget.getPowerRatingWatts());
        powerWidth = Math.max(powerWidth, powerStr.length());

        String statusDisplay = gadget.isOn() ? "RUNNING" : "OFF";
        statusWidth = Math.max(statusWidth, statusDisplay.length());

        String usageTime = gadget.getCurrentUsageTimeFormatted();
        usageWidth = Math.max(usageWidth, usageTime.length());

        String energyStr = String.format("%.3f", gadget.getCurrentTotalEnergyConsumedKWh());
        energyWidth = Math.max(energyStr.length(), energyWidth);
```

```
        // Account for additional rows (current session, timer info)
        if (gadget.isOn() && gadget.getLastOnTime() != null) {
            deviceWidth = Math.max(deviceWidth, "  Current Session:".length());
            String sessionTime = String.format("%.1fh", gadget.getCurrentSessionUsageHours());
            usageWidth = Math.max(usageWidth, sessionTime.length());
        }

        if (gadget.isTimerEnabled()) {
            String timerInfo = buildTimerInfo(gadget, now, timeFormatter);
            String timerDisplay = "  Timer: " + timerInfo;
            deviceWidth = Math.max(deviceWidth, timerDisplay.length());
        }
    }

    return new TableDimensions(
        numWidth,
        Math.max(deviceWidth, 25),
        Math.max(powerWidth, 8),
        Math.max(statusWidth, 9),
        Math.max(usageWidth, 11),
        Math.max(energyWidth, 11)
    );
}
```

Dynamic Sizing Algorithm:
- Content-aware width: Calculates optimal column widths based on actual data
- Minimum guarantees: Ensures columns never get too narrow
- Multi-row consideration: Accounts for session info and timer data
- Professional formatting: Consistent spacing and alignment
- Scalable design: Works with any number of devices

Inner Classes for Table Structure
java

```
private static class TableDimensions {
    final int numWidth, deviceWidth, powerWidth, statusWidth, usageWidth, energyWidth;

    TableDimensions(int numWidth, int deviceWidth, int powerWidth, int statusWidth, int
usageWidth, int energyWidth) {
        this.numWidth = numWidth;
        this.deviceWidth = deviceWidth;
        this.powerWidth = powerWidth;
        this.statusWidth = statusWidth;
        this.usageWidth = usageWidth;
        this.energyWidth = energyWidth;
    }
}

private static class TableFormatStrings {
    final String borderFormat, headerFormat, rowFormat, emptyRowFormat;

    TableFormatStrings(String borderFormat, String headerFormat, String rowFormat, String
emptyRowFormat) {
        this.borderFormat = borderFormat;
        this.headerFormat = headerFormat;
        this.rowFormat = rowFormat;
        this.emptyRowFormat = emptyRowFormat;
    }
}
```

Design Pattern Benefits:
- Inner classes: Encapsulate table formatting logic within service
- Immutable data: Final fields prevent accidental modification
- Type safety: Structured data instead of loose parameters
- Code organization: Related functionality grouped together

Advanced Group Management

Smart Group Administration

java

```
public boolean addPersonToGroup(String memberEmail) {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("[ERROR] Please login first!");
        return false;
    }

    try {
        Customer currentUser = sessionManager.getCurrentUser();
        String currentUserEmail = currentUser.getEmail();

        if (memberEmail.toLowerCase().trim().equals(currentUserEmail.toLowerCase().trim())) {
            System.out.println("[ERROR] You cannot add yourself to the group!");
            return false;
        }

        Customer targetUser = customerService.findCustomerByEmail(memberEmail);
        if (targetUser == null) {
            System.out.println("[ERROR] No registered user found with email: " + memberEmail);
            return false;
        }

        if (currentUser.isGroupMember(memberEmail)) {
            System.out.println("[ERROR] " + memberEmail + " is already in your group!");
            return false;
        }

        // Auto-assign group creator if not set
        if (currentUser.getGroupCreator() == null) {
            currentUser.setGroupCreator(currentUserEmail);
        }
        if (targetUser.getGroupCreator() == null) {
            targetUser.setGroupCreator(currentUserEmail);
        }

        currentUser.addGroupMember(memberEmail);
        targetUser.addGroupMember(currentUserEmail);

        boolean currentUserUpdated = customerService.updateCustomer(currentUser);
        boolean targetUserUpdated = customerService.updateCustomer(targetUser);

        if (currentUserUpdated && targetUserUpdated) {
            sessionManager.updateCurrentUser(currentUser);
            System.out.println("[SUCCESS] " + memberEmail + " has been added to your group!");
            System.out.println("[INFO] Both you and " + memberEmail + " can now see each other's
devices.");
            System.out.println("[INFO] " + memberEmail + " can also see your devices when they
login.");
            System.out.println("[INFO] Group size is now: " + currentUser.getGroupSize() + "
member(s)");
            return true;
        } else {
            // Rollback mechanism for failed updates
            System.out.println("[ERROR] Failed to update group membership in database!");
            if (currentUserUpdated && !targetUserUpdated) {
                currentUser.removeGroupMember(memberEmail);
                customerService.updateCustomer(currentUser);
            }
            if (!currentUserUpdated && targetUserUpdated) {
                targetUser.removeGroupMember(currentUserEmail);
                customerService.updateCustomer(targetUser);
            }
            return false;
        }

    } catch (Exception e) {
        System.out.println("[ERROR] Error adding person to group: " + e.getMessage());
        return false;
```

```
    }
}
```

Sophisticated Group Management Features:
- Bidirectional relationships: Both users added to each other's groups
- Auto-admin assignment: Automatically assigns group creator role
- Transactional updates: Both database updates must succeed
- Rollback mechanism: Reverts changes if any update fails
- Comprehensive validation: Prevents self-addition and duplicates
- User existence verification: Ensures target user is registered

Service Integration and Orchestration
Energy Management Integration
java

```
public void showEnergyReport() {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("[ERROR] Please login first!");
        return;
    }

    Customer currentUser = sessionManager.getCurrentUser();
    EnergyManagementService.EnergyReport report =
energyService.generateEnergyReport(currentUser);

    System.out.println("\n=== " + report.getReportPeriod() + " ===");
    System.out.printf("Total Energy Consumption: %.2f kWh\n", report.getTotalEnergyKWh());
    System.out.printf("Total Cost: Rs. %.2f\n", report.getTotalCostRupees());

    energyService.displayDeviceEnergyUsage(currentUser);
    System.out.println(energyService.getSlabBreakdown(report.getTotalEnergyKWh()));
    System.out.println(energyService.getEnergyEfficiencyTips(report.getTotalEnergyKWh()));
}


 Device Health Monitoring Integration

java
public void showDeviceHealthReport() {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("[ERROR] Please login first!");
        return;
    }

    Customer currentUser = sessionManager.getCurrentUser();
    DeviceHealthService.SystemHealthReport report =
deviceHealthService.generateHealthReport(currentUser);
    deviceHealthService.displayHealthReport(report);
}

public String getSystemHealthSummary() {
    if (!sessionManager.isLoggedIn()) {
        return "Not logged in";
    }

    Customer currentUser = sessionManager.getCurrentUser();
    return deviceHealthService.getHealthSummary(currentUser);
}
```

Timer and Calendar Integration
java

```
public boolean scheduleDeviceTimer(String deviceType, String roomName, String action, String
dateTime) {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("[ERROR] Please login first!");
        return false;
    }
```

```
    try {
        Customer currentUser = sessionManager.getCurrentUser();
        LocalDateTime scheduledTime = timerService.parseDateTime(dateTime);

        return timerService.scheduleDeviceTimer(currentUser, deviceType, roomName, action,
scheduledTime);
    } catch (Exception e) {
        System.out.println("[ERROR] Invalid date format! " + timerService.getTimerHelp());
        return false;
    }
}

public boolean createCalendarEvent(String title, String description, String startDateTime,
String endDateTime, String eventType) {
    if (!sessionManager.isLoggedIn()) {
        System.out.println("[ERROR] Please login first!");
        return false;
    }

    try {
        Customer currentUser = sessionManager.getCurrentUser();
        LocalDateTime startTime = timerService.parseDateTime(startDateTime);
        LocalDateTime endTime = timerService.parseDateTime(endDateTime);

        return calendarService.createEvent(currentUser.getEmail(), title, description,
startTime, endTime, eventType);
    } catch (Exception e) {
        System.out.println("[ERROR] Invalid date format! Use DD-MM-YYYY HH:MM");
        return false;
    }
}
```

Service Orchestration Benefits:
- Unified API: Single entry point for all smart home functionality
- Session awareness: All operations respect user login state
- Error handling: Consistent error messages and exception handling
- Service coordination: Proper delegation to specialized services
- Data flow management: Ensures data consistency across services

Advanced Features Summary

🎭 Facade Pattern Implementation
- Single entry point: Unified interface for complex smart home system
- Complexity hiding: Shields users from multiple service interactions
- Service coordination: Orchestrates interactions between 9 different services
- Consistent API: Uniform method signatures and error handling

🔄 Service Orchestration
- Dependency management: Proper initialization of all required services
- Session integration: Seamless user state management across all operations
- Transaction coordination: Ensures data consistency across service boundaries
- Error propagation: Comprehensive error handling with user-friendly messages

📊 Professional UI Support
- Dynamic table formatting: Auto-sizing tables based on content
- Group-aware displays: Different layouts for individual vs. group users
- Real-time updates: Session synchronization after state changes
- Rich information display: Timer countdowns, session info, health summaries

🛡️ Security and Validation

- Session validation: All operations check login status
- Permission enforcement: Respects device ownership and access rights
- Input validation: Comprehensive parameter checking
- Rollback mechanisms: Transaction safety with automatic rollback

Summary for Beginners

This `SmartHomeService` class demonstrates enterprise-grade architecture patterns:

🎯 Why This Class Exists:
- System coordination: Orchestrates complex interactions between multiple services
- User experience: Provides simple, unified interface for complex functionality
- Business logic: Implements complete smart home workflows and processes
- Maintainability: Centralizes system orchestration for easier maintenance

🏗️ Design Patterns Used:
- Facade Pattern: Simplifies complex subsystem interactions
- Dependency Injection: Manages service dependencies through constructor
- Session Management: Maintains user state across operations
- Inner Classes: Encapsulates related functionality (table formatting)

💡 Real-World Applications:
- Enterprise applications: Central business logic coordination
- Microservices orchestration: Service composition and workflow management
- API gateway patterns: Unified interface for multiple backend services
- System integration: Coordinating multiple specialized subsystems

📚 Programming Concepts You Learned:
- Service composition: Building complex systems from simpler services
- Facade pattern: Providing simplified interfaces to complex subsystems
- Session management: Maintaining user state across multiple operations
- Professional formatting: Dynamic table sizing and presentation
- Transaction management: Ensuring data consistency across operations
- Error handling: Comprehensive exception management with rollback
- User experience design: Consistent messaging and feedback patterns
- Permission systems: Role-based access control implementation

The SmartHomeService shows how modern applications implement system orchestration, user experience optimization, and enterprise-grade architecture to create sophisticated, maintainable software systems!

TimerService.java - Automated Device Scheduling

File Location: `src/main/java/com/smarthome/service/TimerService.java`
Overview
The TimerService class is like a smart personal assistant that never sleeps, constantly watching the clock and automatically controlling your smart home devices at precisely the right times. It acts as an intelligent scheduler that can turn your AC on before you arrive home, start the geyser for morning hot water, or turn off lights late at night - all without any manual intervention. This class implements advanced background processing with thread-safe operations, real-time countdown displays, and automatic cleanup of expired timers. Think of it as having a reliable butler who never forgets to perform scheduled tasks and provides real-time updates on what's coming next.

Package Declaration and Imports
java

```
package com.smarthome.service;

import com.smarthome.model.Customer;
import com.smarthome.model.Gadget;
import com.smarthome.util.SessionManager;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
```

Explanation:
- Service package: Business logic layer for automated device scheduling
- Model imports: Customer and Gadget for device and user management
- SessionManager: Access to current user session information
- Time handling: LocalDateTime, DateTimeFormatter, ChronoUnit for advanced time operations
- Collections: ArrayList and List for managing timer tasks
- Concurrency imports: Advanced threading for background task execution
- ScheduledExecutorService: Professional-grade task scheduling framework
- TimeUnit: Time interval specifications for scheduler

Advanced Threading Concepts:
- `Executors` - Factory for creating thread pools
- `ScheduledExecutorService` - Interface for delayed and periodic task execution
- `TimeUnit` - Time granularity specification (seconds, minutes, hours)

Advanced Singleton Pattern with Dependency Injection
Thread-Safe Singleton Implementation

java

```
public class TimerService {

    private final ScheduledExecutorService scheduler;
    private final CustomerService customerService;
    private static TimerService instance;

    private TimerService(CustomerService customerService) {
        this.scheduler = Executors.newScheduledThreadPool(5);
        this.customerService = customerService;
        startTimerMonitoring();
    }

    public static synchronized TimerService getInstance(CustomerService customerService) {
        if (instance == null) {
            instance = new TimerService(customerService);
        }
        return instance;
    }
```

Advanced Singleton Features:

Feature	Implementation	Purpose
Dependency Injection	Constructor takes CustomerService	Service needs database access for updates
Thread Pool	`newScheduledThreadPool(5)`	5 background threads for concurrent timer execution

Auto-start	`startTimerMonitoring()` in constructor	Begins monitoring immediately upon creation
Thread Safety	`synchronized` keyword	Prevents race conditions in multi-threaded environment

Thread Pool Benefits:
- Concurrent execution: Multiple timers can execute simultaneously
- Resource management: Limited to 5 threads prevents system overload
- Reusable threads: Efficient resource utilization
- Automatic cleanup: Framework handles thread lifecycle

Inner Class for Timer Task Data
TimerTask Structure

java
```
public static class TimerTask {
    private final String deviceType;
    private final String roomName;
    private final String action;
    private final LocalDateTime scheduledTime;
    private final String userEmail;

    public TimerTask(String deviceType, String roomName, String action, LocalDateTime
scheduledTime, String userEmail) {
        this.deviceType = deviceType;
        this.roomName = roomName;
        this.action = action;
        this.scheduledTime = scheduledTime;
        this.userEmail = userEmail;
    }
}
```

Timer Task Components:

Field	Type	Purpose	Example
deviceType	String	What device to control	"AC", "TV", "LIGHT"
roomName	String	Where the device is located	"Living Room", "Bedroom"
action	String	What action to perform	"ON", "OFF"
scheduledTime	LocalDateTime	When to execute	2024-01-15 18:30:00
userEmail	String	Who owns the timer	"john@family.com"

Design Pattern Benefits:
- Immutable data: All fields are final, preventing accidental modification
- Clear encapsulation: Related timer information grouped together
- Type safety: Structured data instead of loose parameters
- Static inner class: Can be used independently of TimerService instance

Intelligent Timer Scheduling
Comprehensive Timer Creation

java
```
public boolean scheduleDeviceTimer(Customer customer, String deviceType, String roomName,
                                   String action, LocalDateTime scheduledTime) {
    try {
        Gadget device = customer.findGadget(deviceType, roomName);
        if (device == null) {
            System.out.println("[ERROR] Device not found: " + deviceType + " in " + roomName);
            return false;
        }
```

```
        LocalDateTime now = LocalDateTime.now();
        if (scheduledTime.isBefore(now)) {
            System.out.printf("[ERROR] Cannot schedule timer for past time!\n");
            System.out.printf("Current time: %s\n", now.format(DateTimeFormatter.ofPattern("dd-
MM-yyyy HH:mm")));
            System.out.printf("Requested time: %s\n",
scheduledTime.format(DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm")));
            return false;
        }

        if (scheduledTime.isBefore(now.plusMinutes(1))) {
            System.out.printf("[ERROR] Timer must be scheduled at least 1 minute in the
future!\n");
            System.out.printf("Minimum allowed time: %s\n",
now.plusMinutes(1).format(DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm")));
            return false;
        }
```

Smart Validation Process:
1. Device existence check: Ensures target device exists in user's account
2. Past time prevention: Blocks scheduling for times that have already passed
3. Minimum delay enforcement: Requires at least 1 minute advance scheduling
4. User-friendly error messages: Clear explanations with specific times shown

Time Validation Logic:
- `scheduledTime.isBefore(now)` - Prevents time travel scenarios
- `now.plusMinutes(1)` - Ensures reasonable scheduling window
- Professional time formatting for user feedback

Action Processing and Database Updates
java

```
        if (action.equalsIgnoreCase("ON")) {
            device.setScheduledOnTime(scheduledTime);
        } else if (action.equalsIgnoreCase("OFF")) {
            device.setScheduledOffTime(scheduledTime);
        } else {
            System.out.println("[ERROR] Invalid action! Use 'ON' or 'OFF'");
            return false;
        }

        device.setTimerEnabled(true);

        boolean updated = customerService.updateCustomer(customer);
        if (updated) {
            System.out.println("[SUCCESS] Timer scheduled for " + device.getType() + " " +
device.getModel() +
                               " in " + device.getRoomName() + " to turn " + action.toUpperCase()
+
                               " at " + scheduledTime.format(DateTimeFormatter.ofPattern("dd-MM-
yyyy HH:mm")));
            return true;
        } else {
            System.out.println("[ERROR] Failed to save timer schedule!");
            return false;
        }
```

Action Processing Features:
- Case-insensitive actions: Accepts "on", "ON", "On", "off", "OFF", "Off"
- Separate ON/OFF scheduling: Device can have both ON and OFF timers simultaneously
- Timer flag activation: `setTimerEnabled(true)` marks device as having active timers

- Database persistence: Changes saved to permanent storage
- Success confirmation: Detailed confirmation message with all timer details

Professional Timer Display System
Dynamic Table Layout with Real-Time Countdowns

java

```
public void displayScheduledTimers(Customer customer) {
    forceTimerCheck();

    System.out.println("\n=== Scheduled Timers ===");

    List<Gadget> devicesWithTimers = new ArrayList<>();
    for (Gadget device : customer.getGadgets()) {
        if (device.isTimerEnabled() &&
            (device.getScheduledOnTime() != null || device.getScheduledOffTime() != null)) {
            devicesWithTimers.add(device);
        }
    }

    if (devicesWithTimers.isEmpty()) {
        System.out.println("No timers scheduled.");
        return;
    }

    LocalDateTime now = LocalDateTime.now();

    System.out.println("+----+-------------------------+--------+------------------+-----------
-----------+");
    System.out.printf("| %-2s | %-23s | %-6s | %-17s | %-20s |\n",
                      "", "Device", "Action", "Scheduled Time", "Status");
    System.out.println("+----+-------------------------+--------+------------------+-----------
-----------+");
```

Smart Display Features:
- Pre-execution check: `forceTimerCheck()` ensures display shows current state
- Filter active timers: Only shows devices with actual timer schedules
- Professional table format: Fixed-width columns with borders
- Real-time data: Current time captured for countdown calculations

Advanced Countdown Calculation System
java

```
private String getCountdownString(LocalDateTime now, LocalDateTime scheduledTime) {
    if (scheduledTime.isBefore(now)) {
        long minutesOverdue = ChronoUnit.MINUTES.between(scheduledTime, now);
        if (minutesOverdue <= 10) {
            return "[EXECUTING/DUE]";
        } else {
            return "[EXPIRED]";
        }
    }

    long totalSeconds = ChronoUnit.SECONDS.between(now, scheduledTime);
    long totalMinutes = totalSeconds / 60;
    long seconds = totalSeconds % 60;
    long days = totalMinutes / (24  60);
    long hours = (totalMinutes % (24  60)) / 60;
    long minutes = totalMinutes % 60;

    if (totalMinutes == 0 && seconds <= 60) {
        return String.format("[%ds remaining]", seconds);
    } else if (totalMinutes < 2) {
        return String.format("[%dm %ds remaining]", minutes, seconds);
    } else if (days > 0) {
```

```
        return String.format("[%dd %dh %dm remaining]", days, hours, minutes);
    } else if (hours > 0) {
        return String.format("[%dh %dm remaining]", hours, minutes);
    } else {
        return String.format("[%dm remaining]", minutes);
    }
}
```

Intelligent Countdown Display:

Time Remaining	Display Format	Example
Past due (≤10 min)	[EXECUTING/DUE]	Timer currently executing
Past due (>10 min)	[EXPIRED]	Timer missed and expired
≤60 seconds	[Xs remaining]	[45s remaining]
<2 minutes	[Xm Xs remaining]	[1m 30s remaining]
With days	[Xd Xh Xm remaining]	[2d 5h 30m remaining]
With hours	[Xh Xm remaining]	[3h 45m remaining]
Minutes only	[Xm remaining]	[25m remaining]

Mathematical Calculations:
- `ChronoUnit.SECONDS.between()` - Precise time difference calculation
- `totalMinutes / (24  60)` - Convert minutes to days
- `(totalMinutes % (24  60)) / 60` - Extract hours component
- `totalMinutes % 60` - Extract remaining minutes

Background Timer Execution Engine
Automated Task Monitoring
java

```
private void startTimerMonitoring() {
        scheduler.scheduleAtFixedRate(() -> {
        try {
            checkAndExecuteScheduledTasks();
        } catch (Exception e) {
            System.err.println("Error in timer monitoring: " + e.getMessage());
        }
    }, 0, 10, TimeUnit.SECONDS);
}
```

Background Monitoring Features:
- Fixed-rate scheduling: Executes every 10 seconds precisely
- Immediate start: `0` initial delay means monitoring starts immediately
- Exception handling: Errors don't crash the monitoring system
- Lambda expression: Modern Java functional programming syntax

Threading Concepts:
- scheduleAtFixedRate() - Maintains consistent 10-second intervals
- Daemon-like behavior - Runs continuously in background
- Exception isolation - Individual timer failures don't stop the service

Comprehensive Timer Execution Logic
java

```
private void checkAndExecuteScheduledTasks() {
    LocalDateTime now = LocalDateTime.now();

    try {
        List<Customer> allCustomers = getAllCustomersWithTimers();

        for (Customer customer : allCustomers) {
            boolean customerUpdated = false;

            for (Gadget device : customer.getGadgets()) {
```

```
                if (!device.isTimerEnabled()) continue;

                if (device.getScheduledOnTime() != null) {
                    LocalDateTime scheduledOnTime = device.getScheduledOnTime();

                    if (now.isAfter(scheduledOnTime) || now.isEqual(scheduledOnTime)) {
                        long minutesSinceScheduled = ChronoUnit.MINUTES.between(scheduledOnTime,
now);

                        if (minutesSinceScheduled <= 10) {
                            String previousStatus = device.getStatus();
                            device.turnOn();
                            String newStatus = device.getStatus();

                            device.setScheduledOnTime(null);
                            customerUpdated = true;

                            if (device.getScheduledOffTime() == null) {
                                device.setTimerEnabled(false);
                            }

                            System.out.println("\n[TIMER EXECUTED] " + device.getType() + " " +
device.getModel() +
                                               " in " + device.getRoomName() + " turned ON
automatically");
                            System.out.println("  Scheduled: " +
scheduledOnTime.format(DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm")));
                            System.out.println("  Executed: " +
now.format(DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss")));
                            System.out.println("  Status: " + previousStatus + " → " +
newStatus);
                            System.out.print("\nPress Enter to continue or enter your choice:
");

                        } else {
                            device.setScheduledOnTime(null);
                            customerUpdated = true;

                            if (device.getScheduledOffTime() == null) {
                                device.setTimerEnabled(false);
                            }

                            System.out.println("[TIMER EXPIRED] Old ON timer removed for " +
                                               device.getType() + " in " + device.getRoomName());
                        }
                    }
                }
```

Smart Execution Logic:

1. Grace Period Handling

- 10-minute execution window: Timers execute within 10 minutes of scheduled time
- Automatic cleanup: Timers older than 10 minutes are automatically removed
- Status tracking: Records previous and new device states

2. Timer State Management

- Individual timer removal: Only clears the executed timer (ON or OFF)
- Smart flag management: Only disables `timerEnabled` when no timers remain
- Database updates: Flags `customerUpdated` for batch saving

3. User Feedback System

- Detailed execution reports: Shows scheduled vs actual execution times
- Status change tracking: "OFF → ON" format shows state transitions
- Interactive prompts: Maintains user interface responsiveness

Advanced Timer Management Features

Smart Timer Cancellation

java

```
public boolean cancelTimer(Customer customer, String deviceType, String roomName, String action)
{
    try {
        Gadget device = customer.findGadget(deviceType, roomName);
        if (device == null) {
            System.out.println("[ERROR] Device not found: " + deviceType + " in " + roomName);
            return false;
        }

        if (action.equalsIgnoreCase("ON")) {
            device.setScheduledOnTime(null);
        } else if (action.equalsIgnoreCase("OFF")) {
            device.setScheduledOffTime(null);
        } else {
            System.out.println("[ERROR] Invalid action! Use 'ON' or 'OFF'");
            return false;
        }

        if (device.getScheduledOnTime() == null && device.getScheduledOffTime() == null) {
            device.setTimerEnabled(false);
        }

        boolean updated = customerService.updateCustomer(customer);
        if (updated) {
            System.out.println("[SUCCESS] Timer cancelled for " + device.getType() + " " +
device.getModel() +
                               " in " + device.getRoomName() + " (" + action.toUpperCase() + "
timer)");
            return true;
        } else {
            System.out.println("[ERROR] Failed to cancel timer!");
            return false;
        }
    }
}
```

Cancellation Features:

- Selective cancellation: Can cancel individual ON or OFF timers
- Smart flag management: Only disables timer flag when all timers are gone
- Database persistence: Saves cancellation to permanent storage
- Confirmation feedback: Clear success/failure messages

Date/Time Parsing and Validation

java

```
public LocalDateTime parseDateTime(String dateTimeStr) throws DateTimeParseException {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
    return LocalDateTime.parse(dateTimeStr, formatter);
}

public String getTimerHelp() {
    StringBuilder help = new StringBuilder();
    help.append("\n=== Timer Scheduling Help ===\n");
    help.append("Date Format: DD-MM-YYYY HH:MM (24-hour format)\n");
    help.append("Examples:\n");
    help.append("- 25-12-2024 18:30 (Christmas evening 6:30 PM)\n");
    help.append("- 01-01-2025 00:00 (New Year midnight)\n");
    help.append("- 15-03-2024 07:00 (March 15, 7:00 AM)\n");
    help.append("\nActions: ON or OFF\n");
    help.append("\nUsage Tips:\n");
    help.append("- Schedule AC to turn ON before you arrive home\n");
    help.append("- Set Geyser timers for morning hot water\n");
    help.append("- Auto-turn OFF lights late at night\n");
    help.append("- Schedule devices during off-peak electricity hours\n");

    return help.toString();
```

```
}
```

User-Friendly Features:
- Consistent date format: DD-MM-YYYY HH:MM (European/Indian format)
- 24-hour time: Eliminates AM/PM confusion
- Practical examples: Real-world scheduling scenarios
- Energy optimization tips: Suggests off-peak usage patterns

Resource Management and Cleanup
Graceful Shutdown System
java

```
public void shutdown() {
    if (scheduler != null && !scheduler.isShutdown()) {
        scheduler.shutdown();
        try {
            if (!scheduler.awaitTermination(60, TimeUnit.SECONDS)) {
                scheduler.shutdownNow();
            }
        } catch (InterruptedException e) {
            scheduler.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}
```

Professional Shutdown Process:
1. Graceful shutdown: `scheduler.shutdown()` allows running tasks to complete
2. 60-second wait: `awaitTermination(60, TimeUnit.SECONDS)` waits for completion
3. Forced shutdown: `shutdownNow()` if graceful shutdown times out
4. Thread interruption handling: Properly handles `InterruptedException`
5. Thread status restoration: `Thread.currentThread().interrupt()` preserves interrupt status

Resource Management Benefits:
- No resource leaks: All threads properly cleaned up
- Graceful degradation: Running timers complete before shutdown
- Exception safety: Handles all shutdown scenarios
- Professional practice: Enterprise-grade resource management

Real-World Usage Examples
Example 1: Morning Geyser Automation
java

```
TimerService timerService = TimerService.getInstance(customerService);

// Schedule geyser to turn ON at 6:00 AM for morning hot water
Customer customer = getLoggedInCustomer();
LocalDateTime morningTime =
LocalDateTime.now().plusDays(1).withHour(6).withMinute(0).withSecond(0);

boolean success = timerService.scheduleDeviceTimer(
    customer,
    "GEYSER",
    "Master Bathroom",
    "ON",
    morningTime
);

if (success) {
    System.out.println("Geyser will automatically turn ON tomorrow at 6:00 AM");

    // Schedule geyser to turn OFF after 1 hour to save energy
    LocalDateTime offTime = morningTime.plusHours(1);
    timerService.scheduleDeviceTimer(customer, "GEYSER", "Master Bathroom", "OFF", offTime);
```

```
    System.out.println("Geyser will automatically turn OFF at 7:00 AM");
}
```

Example 2: Smart AC Pre-Cooling

java

```
// Schedule AC to turn ON 30 minutes before arriving home
LocalDateTime arrivalTime = LocalDateTime.of(2024, 1, 15, 18, 30); // 6:30 PM
LocalDateTime preCoolTime = arrivalTime.minusMinutes(30);           // 6:00 PM

timerService.scheduleDeviceTimer(
    customer,
    "AC",
    "Living Room",
    "ON",
    preCoolTime
);

// Schedule AC to turn OFF at bedtime to save energy
LocalDateTime bedTime = LocalDateTime.of(2024, 1, 15, 23, 0);  // 11:00 PM
timerService.scheduleDeviceTimer(customer, "AC", "Living Room", "OFF", bedTime);
```

Example 3: Automated Light Management

java

```
// Turn ON porch light at sunset
LocalDateTime sunsetTime = LocalDateTime.of(2024, 1, 15, 18, 45);
timerService.scheduleDeviceTimer(customer, "LIGHT", "Porch", "ON", sunsetTime);

// Turn OFF all lights late at night
LocalDateTime lateNight = LocalDateTime.of(2024, 1, 15, 23, 30);
timerService.scheduleDeviceTimer(customer, "LIGHT", "Living Room", "OFF", lateNight);
timerService.scheduleDeviceTimer(customer, "LIGHT", "Kitchen", "OFF", lateNight);
timerService.scheduleDeviceTimer(customer, "LIGHT", "Porch", "OFF", lateNight.plusHours(6)); //
5:30 AM


 Advanced Features Summary
```

🕐 Professional Time Management

- Background monitoring: ScheduledExecutorService runs every 10 seconds
- Precise execution: Grace period handling with 10-minute execution window
- Real-time countdowns: Dynamic countdown displays with multiple time formats
- Smart validation: Past time detection, minimum delay enforcement

🔄 Thread-Safe Operations

- Singleton pattern: Thread-safe instance creation with synchronization
- Concurrent execution: Thread pool allows multiple simultaneous timer executions
- Resource management: Graceful shutdown with proper cleanup
- Exception isolation: Individual timer failures don't affect the service

📊 Professional UI Integration

- Dynamic table formatting: Professional ASCII tables with real-time data
- Status tracking: Before/after state reporting for all timer executions
- Interactive feedback: User prompts maintain interface responsiveness
- Comprehensive help: Built-in documentation with practical examples

🛡️ Robust Error Handling

- Validation layers: Device existence, time validation, action verification
- Graceful degradation: Expired timers automatically cleaned up
- Database transaction safety: Changes saved atomically with rollback capability
- User-friendly error messages: Clear, actionable feedback for all failure scenarios

Summary for Beginners

This `TimerService` class demonstrates advanced concurrency programming and professional task scheduling:

🎯 Why This Class Exists:
- Home automation: Enables hands-free device control based on time schedules
- Energy efficiency: Automates device shutdown during off-peak hours
- Convenience: Eliminates need to manually control devices at specific times
- Reliability: Ensures scheduled tasks execute even when users aren't actively using the system

🧬 Advanced Programming Concepts:
- Singleton with Dependency Injection: Single service instance with required dependencies
- Background Thread Processing: ScheduledExecutorService for continuous monitoring
- Thread-Safe Operations: Synchronized methods and concurrent data handling
- Resource Management: Proper thread pool cleanup and graceful shutdown

💡 Real-World Applications:
- Smart building systems: HVAC scheduling, lighting automation, security timers
- Industrial automation: Equipment startup/shutdown, maintenance scheduling
- IoT device management: Automated device control across distributed systems
- Energy management systems: Load balancing, peak-hour avoidance

📚 Programming Concepts You Learned:
- ScheduledExecutorService: Professional task scheduling framework
- Thread-safe Singleton: Concurrent access patterns with synchronization
- Lambda expressions: Modern Java functional programming
- Time manipulation: LocalDateTime operations and ChronoUnit calculations
- Background processing: Non-blocking task execution
- Resource cleanup: Proper thread pool shutdown and exception handling
- Professional formatting: StringBuilder and dynamic table layouts
- Graceful error handling: Comprehensive exception management

The TimerService shows how modern applications implement automated scheduling, background processing, and professional resource management to create reliable, efficient, and user-friendly automation systems!

Programming Concepts Summary

Object-Oriented Programming Concepts

1. Encapsulation
- Private variables with public getters/setters
- Data hiding prevents direct access to internal state
- Controlled access through well-defined methods

2. Constructor Overloading
- Multiple constructors for different creation scenarios
- Default constructor for framework requirements
- Parameterized constructor for data initialization

3. Method Design Patterns
- Validation methods for business logic
- Utility methods for data formatting

- Query methods for data retrieval

Database Integration Concepts

1. Annotations
- `@DynamoDbBean` marks classes as database entities
- `@DynamoDbPartitionKey` defines primary keys
- `@DynamoDbAttribute` customizes field mapping

2. Data Persistence
- Automatic serialization of Java objects to database
- Type mapping between Java and database formats
- Relationship management between related entities

Security Concepts

1. Access Control
- Role-based permissions for different user types
- Granular control at device level
- Audit trails for security monitoring

2. Account Security
- Failed login tracking prevents brute force attacks
- Account locking with automatic unlock
- Password protection with hashing (implied)

Modern Java Features

1. Enums (Type-Safe Constants)
java
```
public enum GadgetStatus {
    ON, OFF
}
// Usage: status = GadgetStatus.ON.name();
```

2. Stream Operations
java
```
// Find matching devices
gadgets.stream()
    .filter(g -> g.getType().equalsIgnoreCase(type))
    .findFirst()
    .orElse(null);
```

3. Time Handling and Calculations
java
```
// Modern date/time API
LocalDateTime.now()
LocalDateTime.now().plusMinutes(minutes)
ChronoUnit.MINUTES.between(startTime, endTime)
```

4. String Formatting
java
```
// Formatted output
```

```
String.format("%dh %02dm", hours, minutes)  // Zero-padded minutes
String.format("%.3f kWh", energy)           // 3 decimal places
String.format("%.1fW", power)               // 1 decimal place
```

5. Method Overriding

java

```
@Override
public boolean equals(Object obj) { ... }   // Custom equality logic
@Override
public int hashCode() { ... }               // Custom hash calculation
@Override
public String toString() { ... }            // Custom string representation
```

Data Structure Concepts

1. Collections

- ArrayList for dynamic lists
- Null safety with defensive programming
- Duplicate prevention with validation

2. Composite Keys

- Multi-field identifiers for uniqueness
- String concatenation for ID generation
- Case-insensitive comparison for user experience

```
atlas-final/
├── dynamodb-local/
│   ├── DynamoDBLocal.jar
│   ├── DynamoDBLocal_lib/
│   │   └── [158+ dependency JAR files and native libraries]
│   ├── LICENSE.txt
│   ├── README.txt
│   ├── shared-local-instance.db
│   ├── start-db.bat
│   └── THIRD-PARTY-LICENSES.txt
└── iot-smart-home-dashboard/
    ├── pom.xml
    ├── src/
    │   ├── main/
    │   │   ├── java/com/smarthome/
    │   │   │   ├── model/
    │   │   │   │   ├── Customer.java
    │   │   │   │   ├── DeletedDeviceEnergyRecord.java
    │   │   │   │   ├── DevicePermission.java
    │   │   │   │   └── Gadget.java
    │   │   │   ├── service/
    │   │   │   │   ├── AlertService.java
    │   │   │   │   ├── CalendarEventService.java
    │   │   │   │   ├── CustomerService.java
    │   │   │   │   ├── DeviceHealthService.java
    │   │   │   │   ├── EnergyManagementService.java
    │   │   │   │   ├── GadgetService.java
    │   │   │   │   ├── SmartHomeService.java
    │   │   │   │   ├── SmartScenesService.java
    │   │   │   │   ├── TimerService.java
    │   │   │   │   └── WeatherService.java
    │   │   │   ├── util/
    │   │   │   │   ├── DynamoDBConfig.java
    │   │   │   │   └── SessionManager.java
    │   │   │   └── SmartHomeDashboard.java
    │   │   └── resources/
    │   │       └── application.properties
    │   └── test/java/com/smarthome/
```

```
        ├── AlertsFunctionalityTest.java
        ├── AlertsLiveDemo.java
        ├── AutoDeleteAlertsTest.java
        ├── CalendarAutomationFixTest.java
        ├── CalendarDateValidationTest.java
        ├── CalendarIntegrationTest.java
        ├── CalendarNotificationTest.java
        ├── CompleteSceneOrderingConsistencyTest.java
        ├── CompleteStateLifecycleTest.java
        ├── ComprehensiveSystemStatusTest.java
        ├── ComprehensiveSystemTest.java
        ├── CorrectedNotificationMessagesTest.java
        ├── DeviceHealthTableTest.java
        ├── DeviceStateRestorationTest.java
        ├── DeviceStatusChangeNotificationTest.java
        ├── EventEndNotificationTest.java
        ├── EventEndTimingTest.java
        ├── ImprovedAutomationTimingTest.java
        ├── ImprovedNotificationUXTest.java
        ├── InDepthAuthenticationTest.java
        ├── InDepthDeviceManagementTest.java
        ├── OnTimeAutomationTest.java
        ├── SceneOrderingVerificationTest.java
        ├── SimplifiedCalendarTest.java
        ├── SmartHomeServiceCoreTest.java
        ├── SmartHomeServiceEnergyTest.java
        ├── SmartHomeServiceGroupTest.java
        ├── SmartHomeServiceIntegrationTest.java
        ├── SmartHomeServiceTimerTest.java
        ├── SmartScenesOrderingTest.java
        └── StreamlinedCalendarCreationTest.java
    └── target/
        ├── classes/
        ├── test-classes/
        ├── surefire-reports/
        ├── maven-archiver/
        ├── maven-status/
        ├── iot-smart-home-dashboard-1.0.0.jar
        └── original-iot-smart-home-dashboard-1.0.0.jar
```